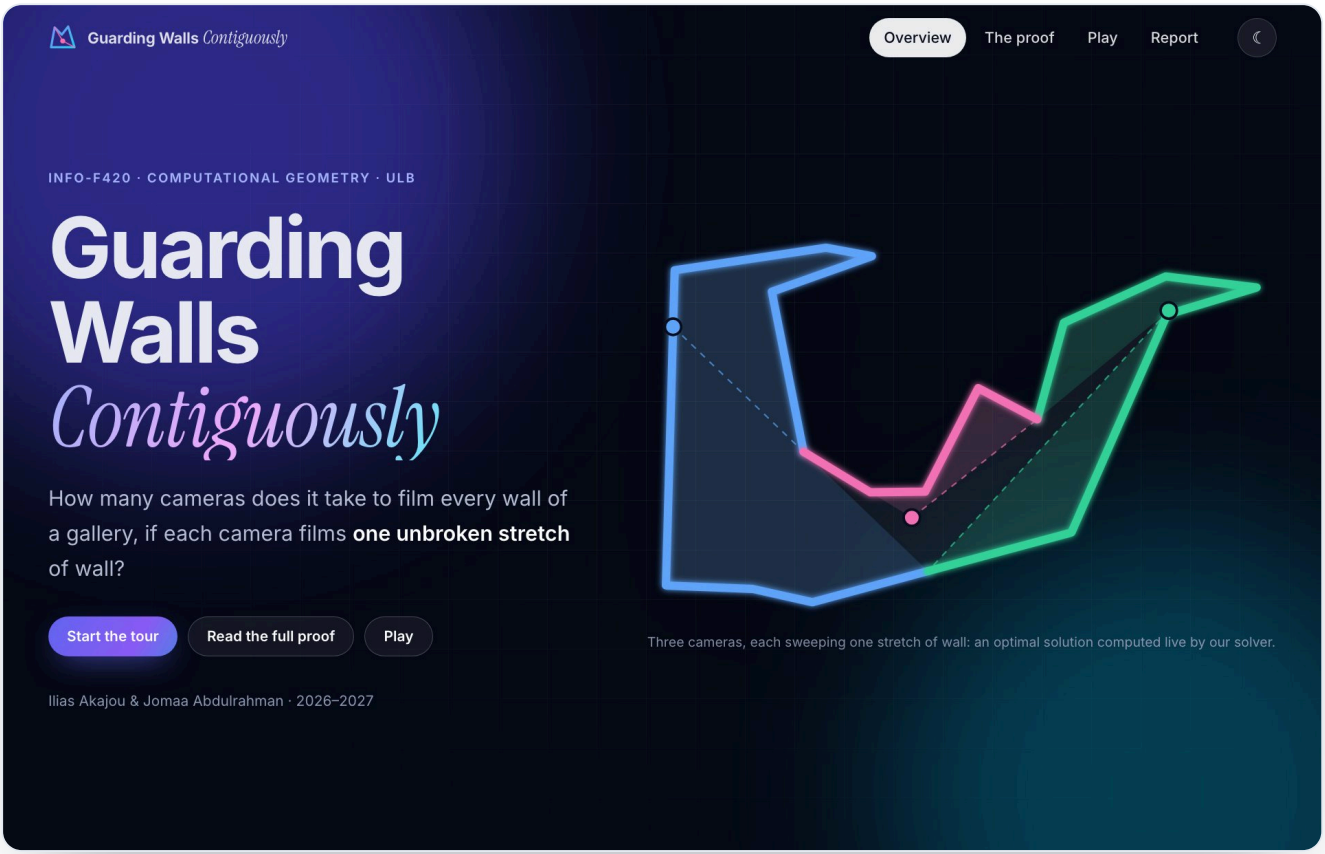


INFO-F420 COMPUTATIONAL GEOMETRY · ULB · 2026–2027

Guarding Walls Contiguously

Dossier complet : le projet, le site, la présentation orale, la preuve en détail, l'implémentation, et comment continuer le travail

Ilias Akajou (520466) · Jomaa Abdulrahman (583629)

Sommaire

A. Le projet en un coup d'œil
B. Visite du site
C. La présentation orale (et les questions probables)
D. La preuve en détail
E. Les figures interactives, une par une
F. L'implémentation et sa validation
G. Continuer le projet

A

Le projet en un coup d'œil

Ce qui a été fait, et comment les remarques du professeur sont prises en compte.

A.1 Le sujet

Le *Contiguous Art Gallery Problem* : dans une galerie (un polygone simple P à n sommets), chaque caméra filme **une seule portion continue de mur**. Combien faut-il de caméras, au minimum, pour filmer tous les murs ? Contrairement à presque toutes les variantes de la galerie d'art (NP-difficiles ou $\exists\mathbb{R}$-complètes), ce problème se résout en temps polynomial. Notre projet explique l'algorithme de de Berg, Conradi, van der Hoog et Staals [5] en $O(kn^2 \log^2 n)$, et résume l'algorithme optimal en $\Theta(n \log n)$ des mêmes auteurs [6].

Hypothèse essentielle : P est un polygone simple (un seul contour qui ne se croise pas, sans trou). Le bord est alors une seule boucle, les plus courts chemins sont uniques et toute courbe fermée dans P n'entoure que des points de P : toutes les preuves en dépendent. Avec des trous, la méthode de Merrild et al. échoue et la complexité du problème est ouverte. Le site le dit sur l'accueil, au chapitre 1 et dans le rapport ; le solveur refuse un polygone non simple.

A.2 Les demandes du professeur et ce que nous avons fait

Remarque	Ce qui est fait
Ne pas passer trop de temps sur le problème classique de la galerie d'art	L'accueil ne garde que la définition (section 01) et pourquoi c'est surprenant (section 02). Aucune ré-explication du théorème de Chvátal (une phrase dans le rapport).
Travailler tous les deux sur la partie 2 et se répartir les explications	La preuve est découpée en 5 chapitres ; chaque chapitre et chaque section affichent « Explained by … ». Répartition en A.4.
Des widgets web qui illustrent les étapes de la preuve	11 figures interactives (une par étape) dans les chapitres, plus 4 animations sur l'accueil et le jeu. Toutes calculées en direct par notre implémentation.

A.3 Ce qui est livré

- **Le site** (React, publiable sur GitHub Pages) : un accueil = le support de l'oral, la preuve en 5 chapitres, le jeu, le rapport.
- **Le solveur** : l'algorithme de [5] en JavaScript, utilisé par toutes les figures et par le jeu.
- **Les tests** : 1541/1541 checks passed ; bons gardiens : 8647/8647 remplacements trouvés (prop. 4.3, cor. 4.7) ; mauvais gardiens : 1381/1381 (lemme 4.9) ; lemme 6.2 : 5014/5014 égal au calcul direct ; théorème 5.4 : 61/61 déroulés terminés dans U_P.

- **Ce dossier**, plus deux brouillons à compléter : la répartition du travail et le journal IA (docs/).
- **L'archive** à envoyer : sources, aperçu du site en un seul fichier HTML, documents.

A.4 Répartition des explications

Partie	Contenu	Explique
Accueil 01–02	Le problème, pourquoi c'est surprenant	Ilias
Chapitre 1	Noyaux de visibilité, glouton, borne $k+1$	Ilias
Chapitre 2	Bons/mauvais gardiens, lemmes de remplacement	Ilias
Chapitre 3	G_P, U_P, théorème 5.4	Jomaa
Chapitre 4	Lemme 6.2, lemme 6.8, algorithme complet	Jomaa
Chapitre 5	Algorithme optimal $\Theta(n \log n)$	Jomaa
Jeu, rapport	—	les deux

Cette répartition est une proposition : elle se change dans `src/sections/blocks.jsx` (noms) et dans `src/pages/ChapterPage.jsx` / `src/sections/chapters.jsx` (attributs `author`).

B

Visite du site

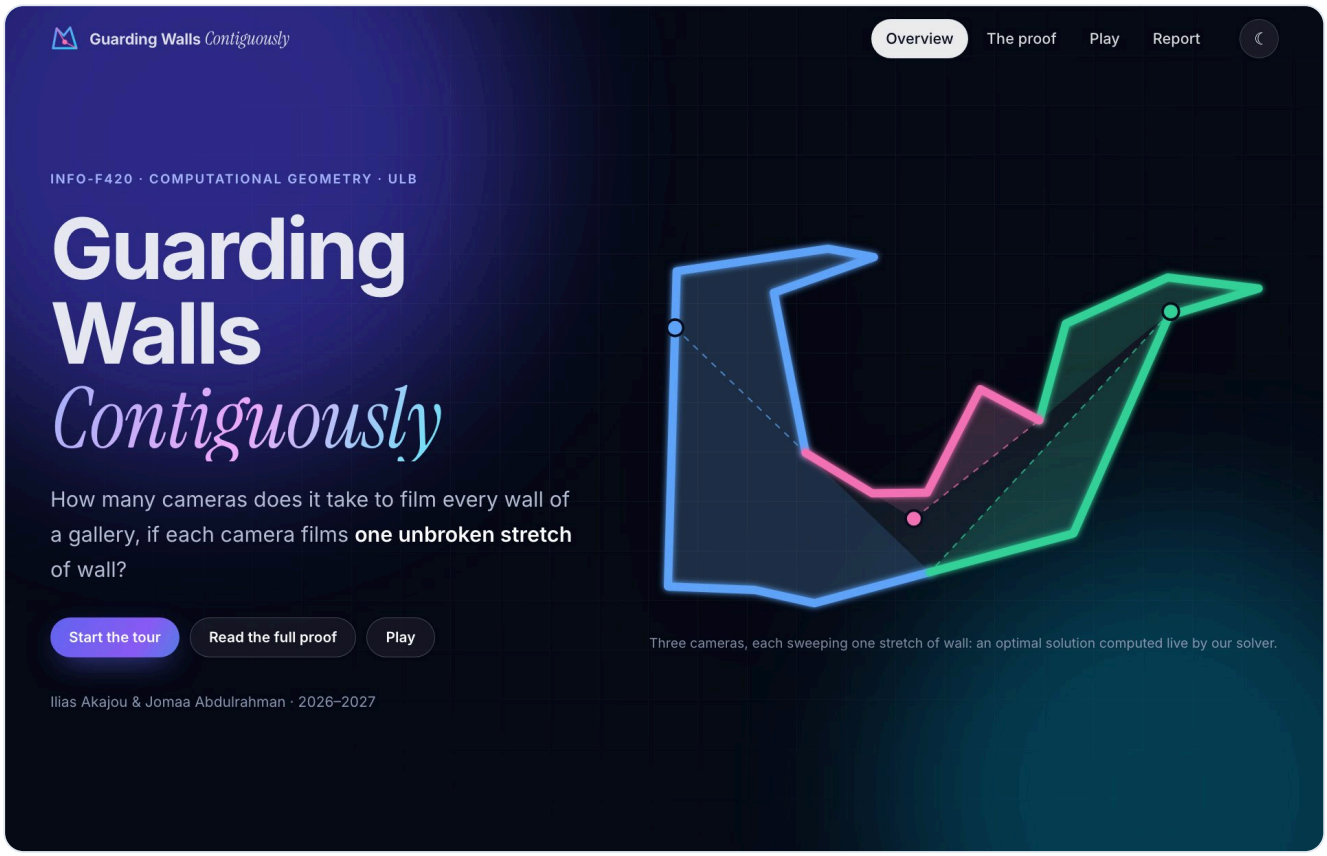
Une page d'accueil courte pour l'oral, et des pages de détail pour tout le reste.

B.1 Plan du site

Adresse	Page	Contenu
`#/`	Overview	Le support de l'oral : 6 sections courtes, 1 figure interactive, 4 animations
`#/proof`	The proof	Carte de la preuve et accès aux 5 chapitres
`#/proof/1` … `#/proof/5`	Chapitres	Énoncés, preuves détaillées, figures interactives 2 à 11
`#/play`	Play	Le jeu et le bac à sable
`#/report`	Report	Résumé, contexte, implémentation, équipe, références

Le routage utilise le fragment `#/…` : le site fonctionne tel quel sur GitHub Pages et même ouvert comme simple fichier. Un thème sombre est disponible (bouton ☾).

B.2 L'accueil, section par section



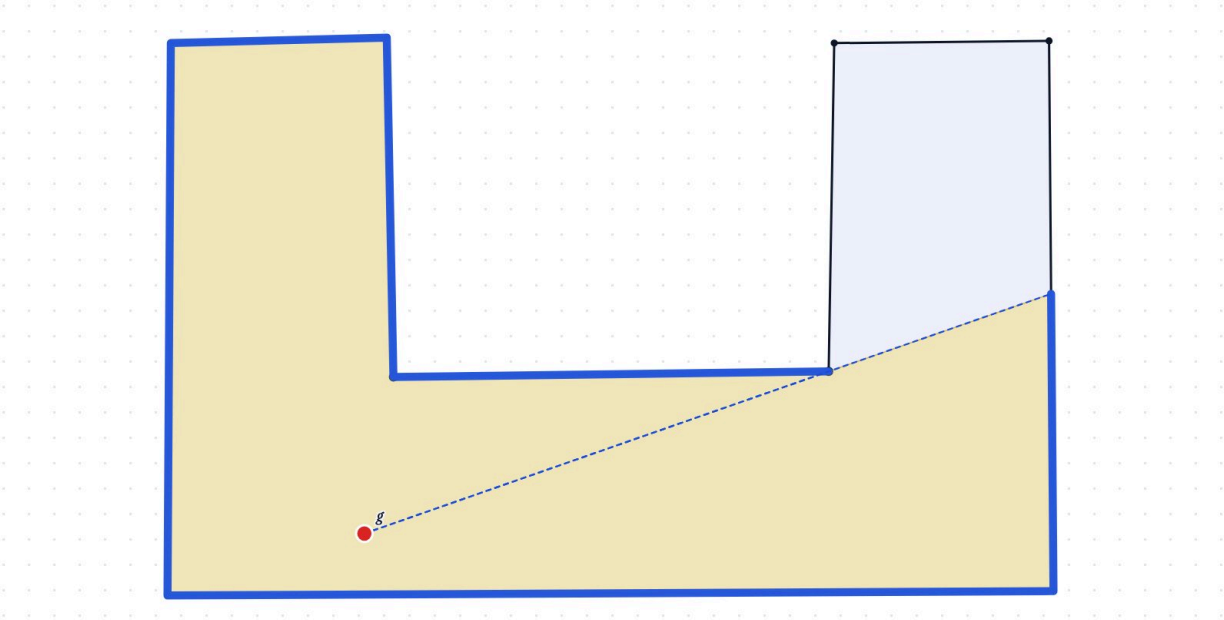
En-tête. Le titre, la question en une phrase, et une animation : trois caméras balaient chacune leur portion de mur. C'est une solution optimale calculée en direct par notre solveur sur le polygone « Atrium ».

01 THE PROBLEM

One camera, *one stretch of wall*

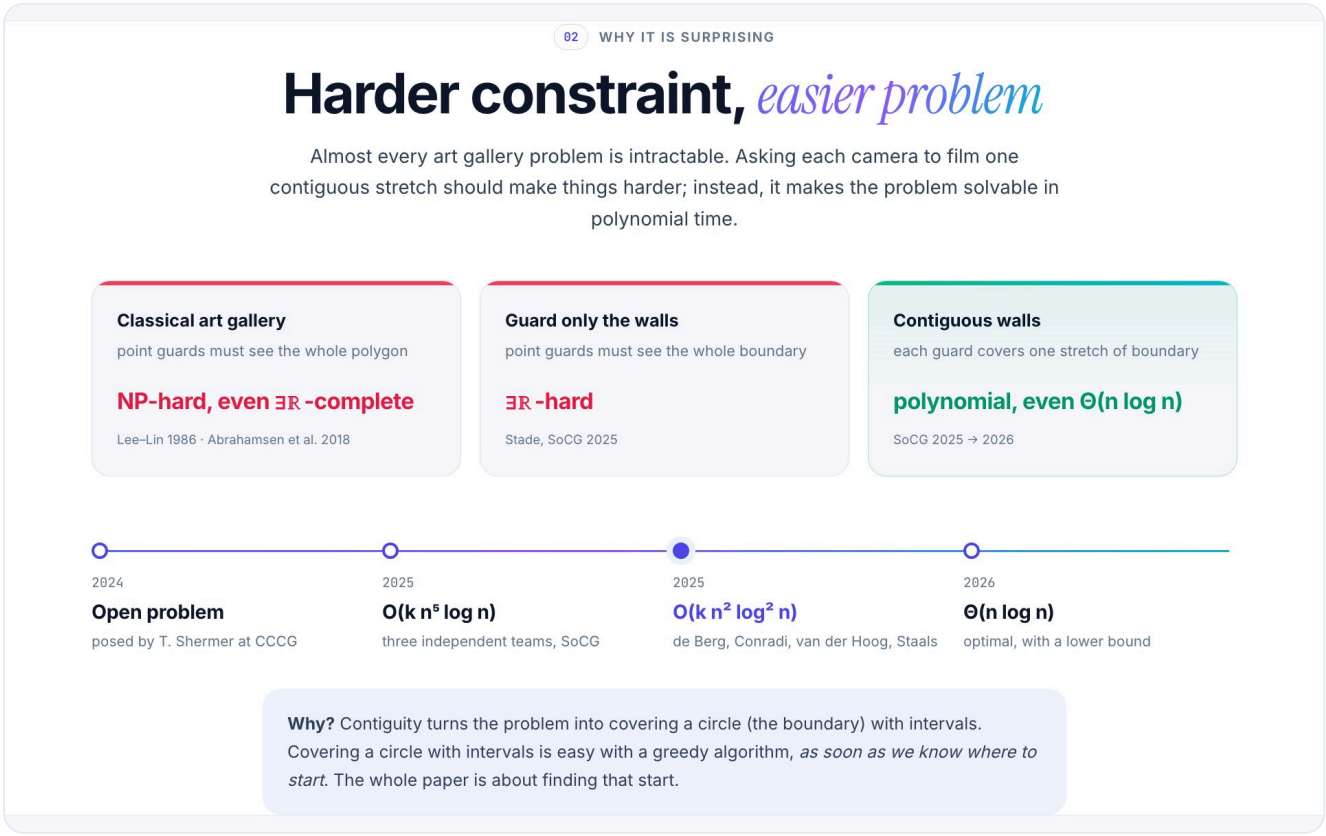
A **contiguous guard** is a point g of the polygon together with a single stretch $[u, v]$ of the boundary that g sees entirely. We want the minimum number k of contiguous guards whose stretches cover all walls.

FIGURE 1 **A contiguous guard** Polygon U-shaped room show an optimal solution

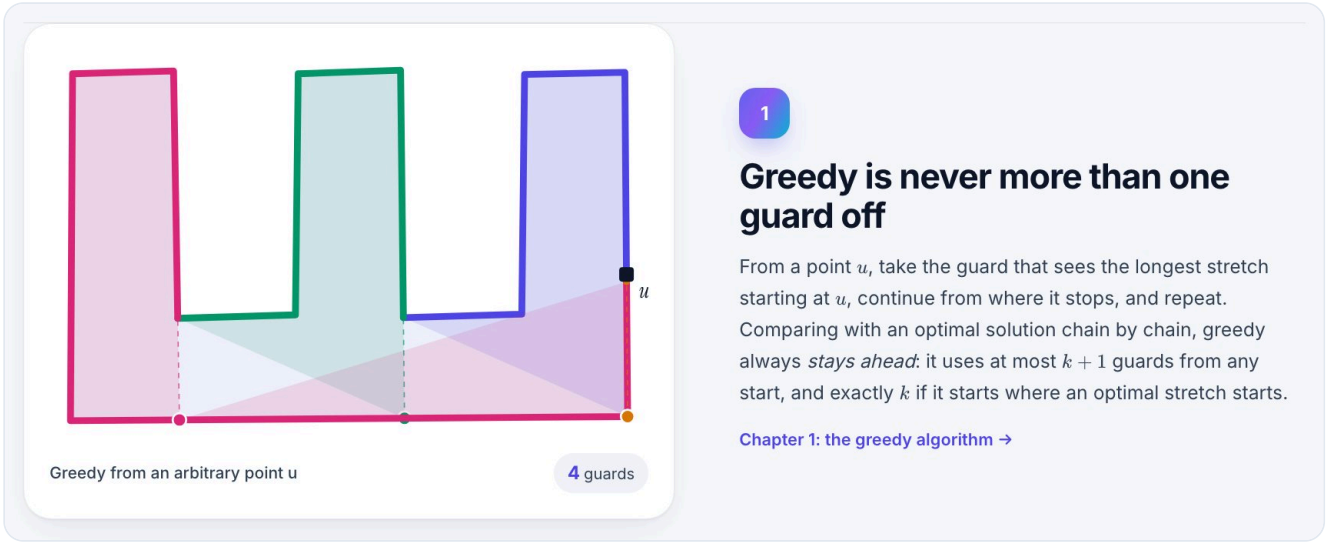


Drag the guard g. The yellow region is its visibility polygon and the light blue pieces of the boundary are the *maximal chains* it sees. A contiguous guard has to pick **one** of them (click on a light blue piece to choose it). Here g sees 1 maximal chain.

01 — Le problème. La définition en deux phrases et la figure 1 : on déplace un point, on voit ce qu'il voit, on choisit une portion.



02 — Pourquoi c'est surprenant. Trois cartes (galerie classique, murs seulement, murs contigus) et une frise animée 2024 → 2026 qui se dessine au défilement.



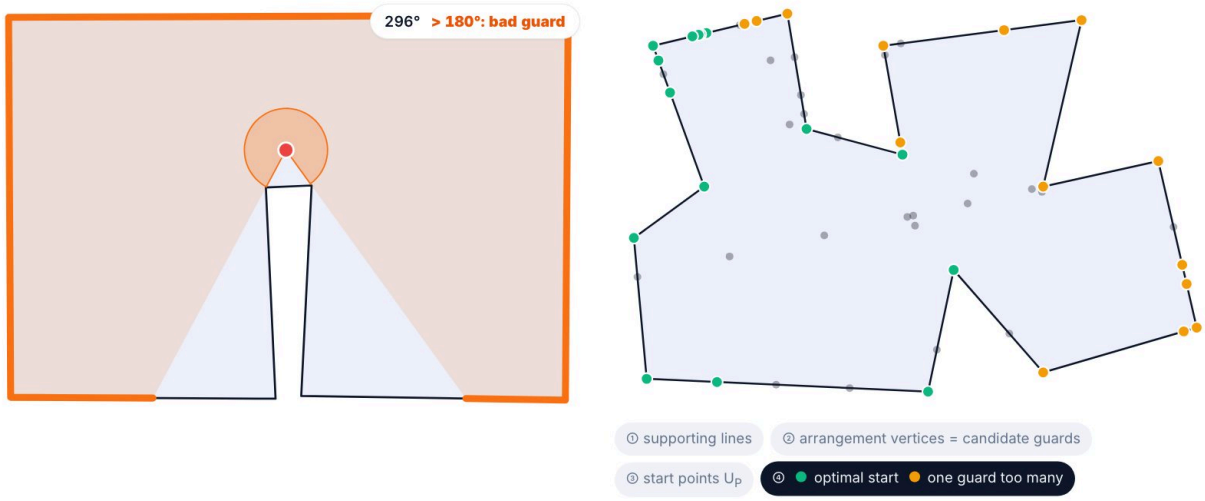
03 — L'algorithme en une phrase, idée 1. Animation : le glouton depuis un point quelconque utilise 4 gardiens ($k+1$), puis depuis le début d'une chaîne optimale 3 gardiens (k).

2 Only $O(n^2)$ starting points matter

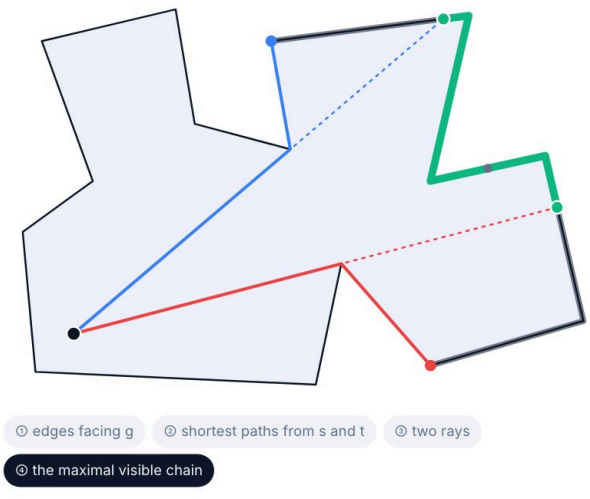
A guard is **bad** if it looks around a corner (it sees its stretch under more than 180°), and **good** otherwise. A good guard can slide to a crossing point of the lines through the edges without losing anything; a solution made only of bad guards can be reshaped to contain a good one.

So some optimal stretch starts at one of the $O(n^2)$ points U_P derived from those crossings (Theorem 5.4).

Chapter 2: good and bad guards → Chapter 3: Theorem 5.4 →



Idée 2. À gauche, un mauvais gardien : le faisceau balaie plus de 180° autour de l'encoche. À droite, les droites support apparaissent, puis les sommets de l'arrangement, puis les départs U_P colorés (vert : départ optimal, orange : un gardien de trop).



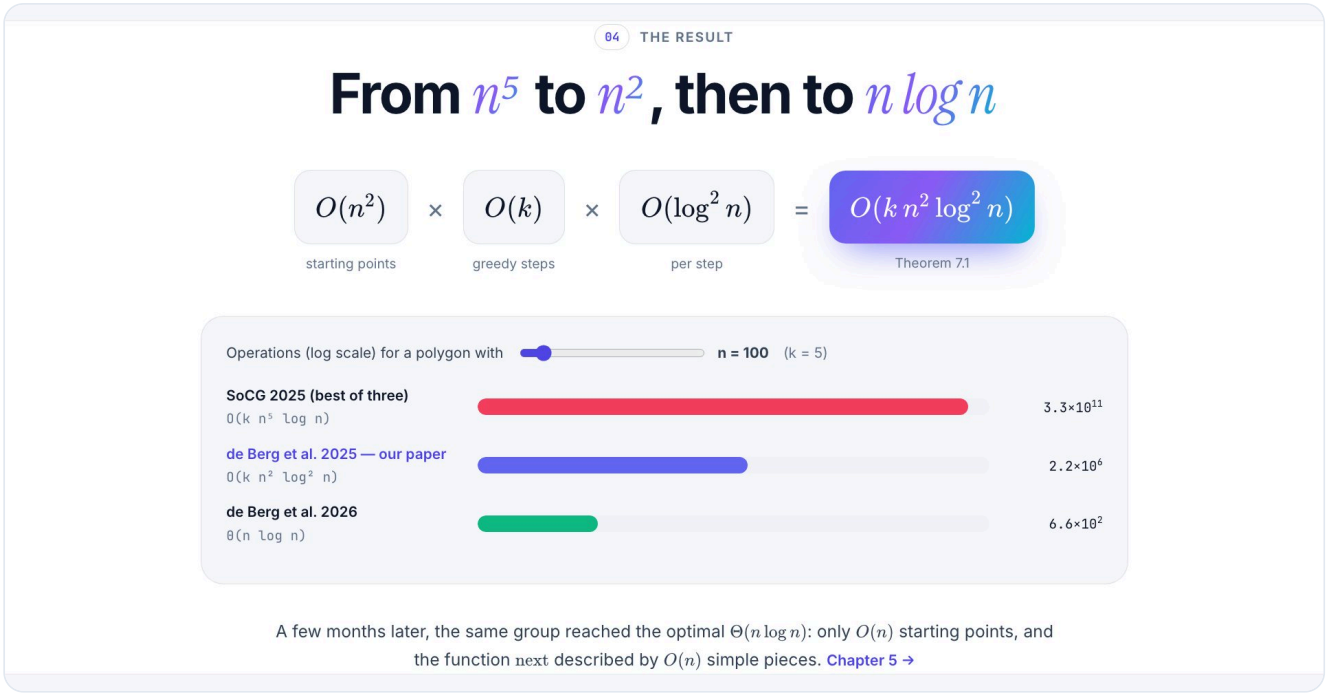
3 Each greedy step takes $O(\log^2 n)$

What a guard sees around a point is found with only **two shortest paths and two rays** (Lemma 6.2): the paths wrap around the obstacles, and the rays that extend them mark where the visible stretch begins and ends.

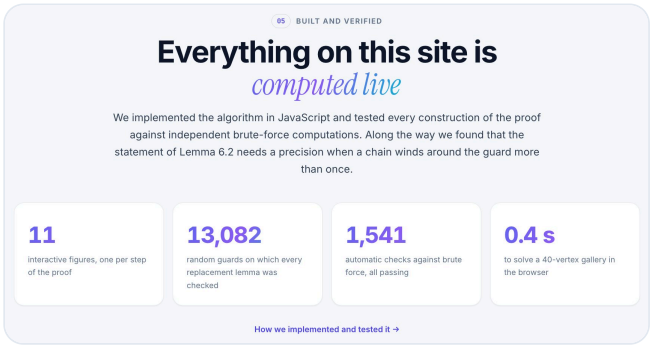
With these structures and a few precomputed cores, one greedy step costs $O(\log^2 n)$, even for bad guards.

Chapter 4: fast greedy steps →

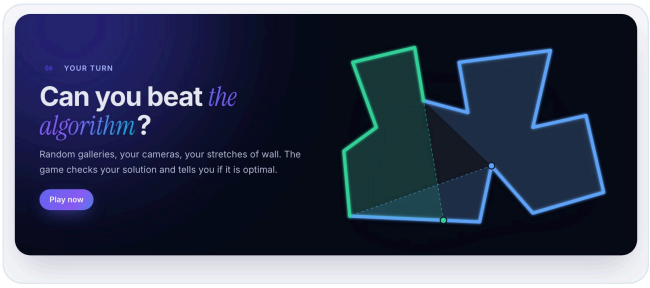
Idée 3. Le lemme 6.2 animé : arêtes vues de face, deux plus courts chemins, deux rayons, puis la chaîne visible maximale.



04 — Le résultat. La formule $O(n^2) \times O(k) \times O(\log^2 n)$ et une « course » des complexités en échelle logarithmique (curseur n).

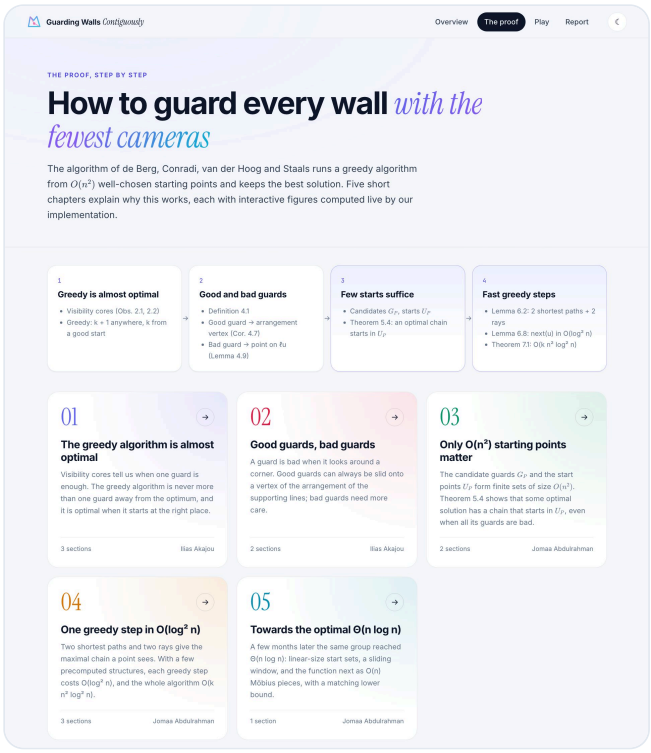


05 — Construit et vérifié. Les chiffres de nos tests (compteurs animés).

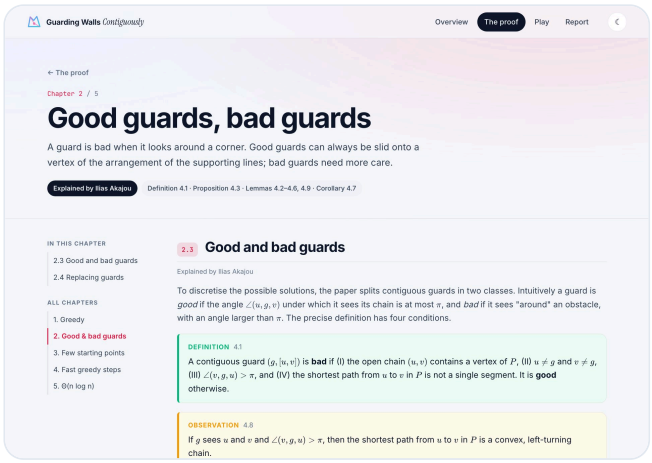


06 — À vous de jouer. Lien vers le jeu, avec une animation.

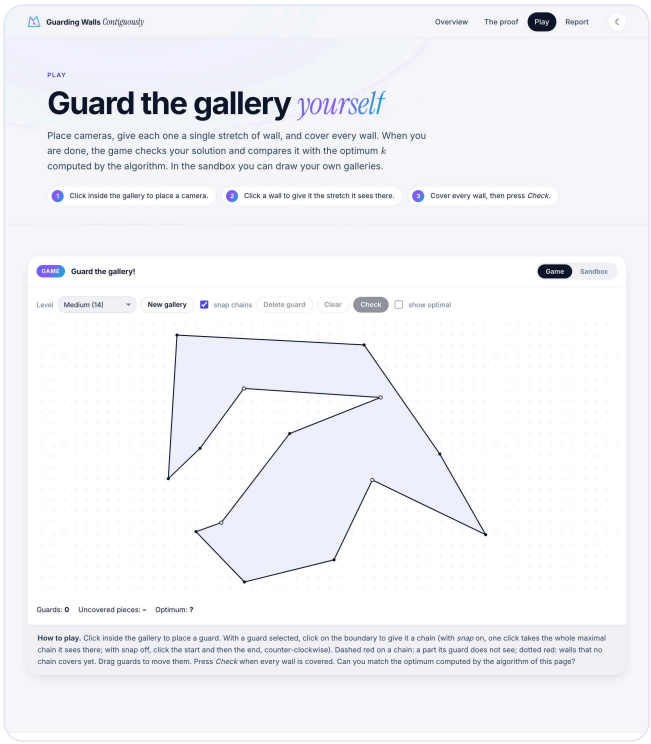
B.3 Les pages de détail



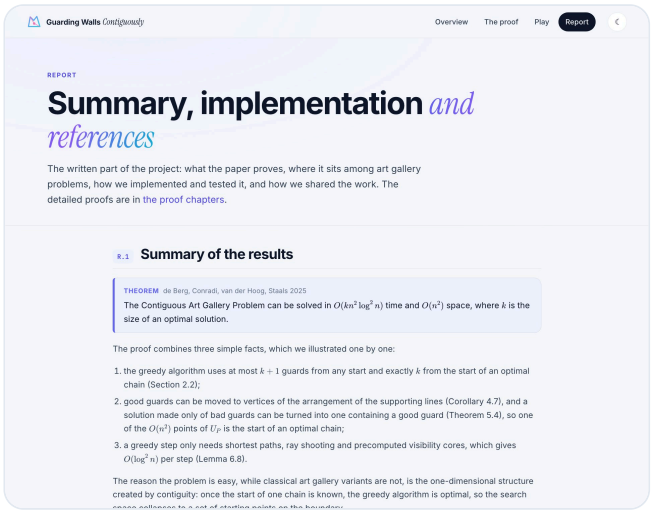
The proof. La carte de la preuve en 4 étapes et les 5 chapitres.



Un chapitre. En-tête coloré, auteur, résultats couverts, sommaire latéral, puis énoncés, preuves et figures.



Play. Le jeu en trois étapes.



Report. Résumé, contexte, implémentation, équipe, références.

B.4 Design et animations

- Typographies : Inter (texte), Instrument Serif (mots en italique dégradé), JetBrains Mono (code). Thème clair et thème sombre.

- Animations : apparition des sections au défilement, changement de page en fondu, caméras qui balaient les murs, frise et barres qui se dessinent, compteurs, étapes de preuve en fondu, confettis quand on trouve l'optimum au jeu.
- Les animations ne tournent que quand elles sont visibles, et sont désactivées si l'ordinateur demande de « réduire les animations ».

C

La présentation orale

L'accueil du site est le support : on le déroule de haut en bas.

C.1 Déroulé (≈ 12–15 minutes)

Section de l'accueil	Qui	Durée	Ce qu'on dit	Geste
En-tête	Ilias	0:30	La question : combien de caméras, si chacune filme une portion continue de mur ?	Laisser tourner l'animation
01 Le problème	Ilias	1:30	Définition du gardien contigu ; un point voit plusieurs morceaux mais n'en garde qu'un.	Bouger g, cocher « optimal »
02 Pourquoi c'est surprenant	Ilias	1:30	Les autres variantes sont dures ; la contiguïté ramène à couvrir un cercle par des intervalles ; frise des résultats.	Faire défiler la frise
03 Idée 1	Ilias	2:00	Le glouton ; « il reste en avance » : au plus $k+1$, et k depuis le bon départ.	Montrer 4 puis 3 gardiens
03 Idée 2	Jomaa	3:00	Bons/mauvais gardiens ; un bon gardien glisse vers un sommet de l'arrangement ; théorème 5.4 : un départ de U_P est optimal.	Ouvrir le chapitre 3, figure 7, dérouler les 4 étapes
03 Idée 3	Jomaa	2:00	Lemme 6.2 : deux plus courts chemins et deux rayons ; un pas glouton en $O(\log^2 n)$.	Laisser tourner l'animation
04 Le résultat	Jomaa	1:30	$O(kn^2 \log^2 n)$, puis $\Theta(n \log n)$ avec borne inférieure.	Bouger le curseur n
05–06	les deux	1:30	Notre implémentation, nos tests, la remarque sur le lemme 6.2 ; démo du jeu.	Page Play, niveau « Easy »

C.2 Questions probables et réponses

Pourquoi le glouton utilise-t-il au plus $k+1$ gardiens ?

Argument « le glouton reste en avance » : on numérote les chaînes optimales à partir de celle qui contient u (indice 0). La i-ème chaîne gloutonne finit au moins aussi loin que la $(i-1)$-ème chaîne optimale, parce que le point où le glouton reprend est dans la i-ème chaîne optimale, dont le gardien voit donc jusqu'à sa fin. Après $k+1$ chaînes on a dépassé la fin de la chaîne 0 après un tour.

Pourquoi ne pas simplement essayer tous les points du bord ?

Il y en a une infinité. Il faut un ensemble fini de départs dont l'un est le début d'une chaîne optimale : c'est U_P ($O(n^2)$ points). Nos expériences montrent qu'en pratique environ la moitié des points de U_P sont de bons départs, mais la garantie vient du théorème 5.4.

Qu'est-ce qu'un mauvais gardien, intuitivement, et pourquoi pose-t-il problème ?

C'est un gardien qui voit sa chaîne sous un angle de plus de π, en regardant autour d'un obstacle : u et v ne se voient pas. Pour un bon gardien, le polygone P_g est à gauche des segments gu et gv, donc tout point de $P_E \cap P_g$ est dans le noyau de P_g et voit la chaîne. Pour un mauvais gardien ce n'est plus vrai, et le remplaçant (lemme 4.9) se trouve sur une droite ℓ_u qui dépend de u : on ne peut pas le précalculer, on le calcule à la volée pendant le glouton.

Comment le théorème 5.4 se débarrasse-t-il des solutions faites uniquement de mauvais gardiens ?

La chaîne d'un mauvais gardien g est maximale, donc (lemme 5.3) le segment de sa fin v vers g passe par un sommet réflexe r que g ne couvre pas. Le gardien g' qui couvre r est forcé dans P_g ; on enlève de sa chaîne la partie déjà vue par g, et le reste est vu sous un angle $\leq \pi$: g' devient bon, et un bon gardien se remplace par un candidat de G_P dont la chaîne commence dans U_P.

Pourquoi les sommets de l'arrangement des droites support ?

Parce que les contraintes de visibilité d'une chaîne sont « être à gauche des droites support de ses arêtes » (obs. 2.1) : la région admissible est un polygone convexe dont les sommets sont des intersections de ces droites. Pousser un bon gardien jusqu'à un sommet de cette région ne lui fait rien perdre (prop. 4.3).

Que calcule exactement le lemme 6.2 et pourquoi deux plus courts chemins ?

La chaîne maximale vue par g contenant m. Ce qui cache le bord à g, ce sont des sommets réflexes ; le plus court chemin de s (début de la zone où g est à gauche des arêtes) vers g s'enroule autour du dernier obstacle, et le rayon qui prolonge sa dernière arête donne la limite de l'ombre u. Idem de l'autre côté avec t pour v.

Avez-vous trouvé quelque chose de faux ou d'imprécis dans l'article ?

Une imprécision dans l'énoncé du lemme 6.2 : quand la chaîne fait plus d'un tour autour de g, un rayon peut toucher $[s,t]$ deux fois ; il faut prendre le point le plus proche de m, comme dans la définition de u^* de la preuve. Sinon, toutes les constructions de l'article ont fonctionné sur plus de 13 000 gardiens aléatoires.

Quelle est la complexité de votre implémentation ?

Polynomiale mais plus élevée que l'article (pas de structures en $O(\log n)$) : graphe de visibilité et Floyd–Warshall en $O(n^3)$, visibilité exacte d'un point en $O(n^2)$–$O(n^3)$, balayage linéaire des arêtes au lieu de la recherche binaire. Sous la seconde pour $n \leq 40$, ce qui suffit pour le jeu et les figures.

Comment savez-vous que votre solveur est correct ?

(1) Chaque gardien proposé vient d'un vrai calcul de visibilité : le solveur ne peut pas tricher sur la couverture. (2) On compare $\mathrm{next}(u)$ à une force brute sur une grille dense de gardiens, et l'optimum à des gloutons depuis 200 départs. (3) Chaque lemme est testé sur des milliers de gardiens aléatoires. Tout passe.

Et si le polygone n'est pas en position générale ?

L'article le suppose. Dans notre code, les droites parallèles n'ont pas d'intersection (ignorées) et les points concourants sont fusionnés en gardant toutes les arêtes qui les définissent. Le jeu accepte donc aussi des polygones orthogonaux dessinés à la main ; les exemples du site sont légèrement perturbés.

Quelle est la différence entre [5] et [6] ?

[5] : $O(n^2)$ départs et un pas glouton en $O(\log^2 n)$, soit $O(kn^2 \log^2 n)$. [6] : $O(n)$ départs (dominateurs réduits), une fenêtre glissante pour calculer next sur tous les départs en $O(n \log n)$, puis la représentation de next par $O(n)$ morceaux de Möbius pour éliminer le facteur k ; plus une borne inférieure $\Omega(n \log n)$ par réduction depuis *set disjointness*.

Pourquoi $\Omega(n \log n)$ et pas $\Omega(n)$?

Parce qu'on peut encoder *set disjointness* (décider si deux ensembles de n nombres sont disjoints, $\Omega(n \log n)$ comparaisons) dans un polygone : chaque élément de B crée une poche gardable seulement depuis un point précis du long mur, et chaque élément de A rend ce point inutilisable.

Comment générez-vous les polygones du jeu ?

Points aléatoires reliés dans un ordre aléatoire, puis « 2-opt » : tant que deux arêtes se croisent on inverse la portion de tournée entre elles (la longueur diminue strictement, donc ça termine). On rejette les polygones avec des angles trop aigus ou des sommets trop proches d'autres arêtes, et les polygones étoilés (trop faciles).

C.3 Pièges à éviter

- Ne pas dire que le glouton est optimal : il l'est seulement depuis un bon départ ; ailleurs il peut utiliser $k+1$ gardiens.
- « Bon » et « mauvais » ne veulent pas dire « utile » et « inutile » : une solution optimale peut ne contenir que des mauvais gardiens (figure 7).
- U_P n'est pas un ensemble de positions de gardiens mais de *points de départ* sur le bord ; les positions candidates sont les sommets de C_P (G_P).
- P_E n'est pas forcément dans P : c'est une intersection de demi-plans du plan entier.

D

La preuve en détail

Toutes les étapes de l'article [5], avec les preuves complètes et des figures calculées par notre code.

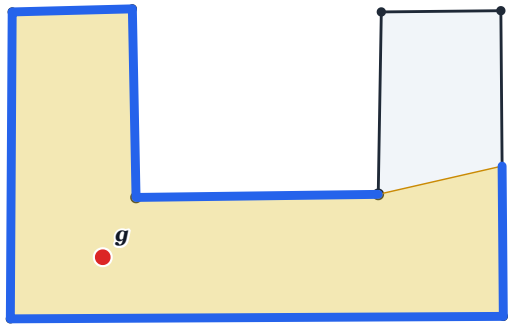
0. Comment lire ce document

Le professeur nous a conseillé de ne pas passer trop de temps sur le problème classique de la galerie d'art (beaucoup de ressources existent) et de **travailler tous les deux sur la partie 2**, en nous répartissant les explications. Le document est donc surtout consacré à la preuve de l'article [5]. Chaque section indique le lemme correspondant de l'article et la figure interactive du site qui l'illustre. La dernière section propose une répartition des sections de la partie 2 entre nous deux (à adapter).

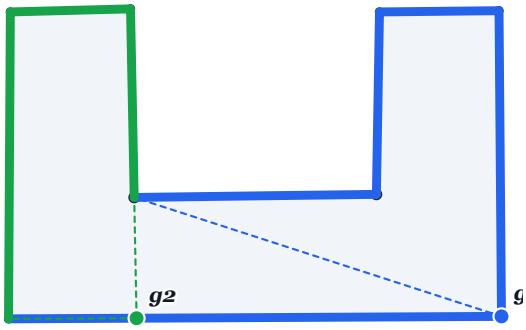
Notations. P est un polygone simple à n sommets $v_0, \dots, v_{n-1}$ donnés dans le sens trigonométrique (anti-horaire). L'arête $e_i = v_i v_{i+1}$ est orientée, donc l'intérieur de P est *à gauche* de chaque arête. Pour deux points u, v du bord ∂P, la *chaîne* $[u, v]$ est la partie du bord parcourue de u à v dans le sens anti-horaire, et (u, v) la même chaîne sans ses extrémités. Un point g *voit* q si le segment gq est contenu dans P ; P est un ensemble **fermé** : un segment qui longe une arête ou qui frôle un sommet réflexe reste « dans » P. Ce détail compte beaucoup dans les preuves (les gardiens candidats sont sur les droites support des arêtes).

1. Le problème (rappel bref)

Définition. Un **gardien contigu** est un couple $(g, [u, v])$ où $g \in P$ et $[u, v]$ est une chaîne dont *chaque* point est vu par g. Le **Contiguous Art Gallery Problem** demande le nombre minimum k de gardiens contigus dont les chaînes recouvrent ∂P. On peut supposer que les chaînes ne se chevauchent pas : elles forment une partition du bord.



Un point g (rouge), sa région de visibilité (jaune) et les chaînes maximales qu'il voit (bleu). Un gardien contigu n'en garde qu'*une*.



Une solution optimale du polygone en U : $k = 2$ gardiens, chacun avec sa chaîne (couleurs).

Pourquoi c'est surprenant. La version classique (gardiens ponctuels qui doivent voir tout le polygone) est NP-difficile [8] et même $\exists\mathbb{R}$-complète [9] ; ne garder que le bord avec des gardiens ponctuels est aussi $\exists\mathbb{R}$-difficile [10]. Ajouter la contrainte de contiguïté semblait rendre le problème plus dur, mais c'est l'inverse : chaque gardien couvre *un intervalle* d'un objet de dimension 1 (le bord), et couvrir un cercle par des intervalles est facile dès qu'on sait quels intervalles utiliser. C'est exactement ce que le glouton exploite.

Année	Référence	Idée	Temps
2024	Shermer (CCCG)	problème posé	ouvert
2024–25	[2], [3], [4] → [1] (SoCG 2025)	glouton répété / $O(n^4)$ candidats / arcs analytiques	$O(kn^5 \log n)$, $O(n^6 \log n)$
2025	**[5] de Berg, Conradi, van der Hoog, Staals**	**glouton depuis $O(n^2)$ départs**	$O(kn^2 \log^2 n)$
2025–26	[6] de Berg, Conradi, van der Hoog, Rotenberg	dominateurs, fenêtre glissante, Möbius	$\Theta(n \log n)$

2. Vue d'ensemble de la preuve

L'algorithme de [5] tient en une phrase : **lancer un algorithme glouton depuis $O(n^2)$ points de départ bien choisis sur le bord, et garder la plus petite solution**. La preuve qu'il est correct et rapide repose sur trois idées.

1. **Le glouton est presque optimal** : depuis n'importe quel point, il utilise au plus $k+1$ gardiens, et exactement k s'il part du début d'une chaîne d'une solution optimale (sections 2.1–2.2).
2. **Peu de départs suffisent** : il existe toujours une solution optimale dont une chaîne commence dans un ensemble explicite U_P de $O(n^2)$ points. L'outil clé est la distinction entre *bons* et *mauvais* gardiens : un bon gardien peut être déplacé sur un sommet de l'arrangement des droites support ; une solution faite uniquement de mauvais gardiens peut être transformée pour contenir un bon gardien (sections 2.3–2.6).
3. **Chaque pas glouton est rapide** : en $O(\log^2 n)$ grâce aux plus courts chemins, au lancer de rayon et à des noyaux de visibilité précalculés (sections 2.7–2.8).

Total : $O(n^2)$ départs $\times$ $O(k)$ pas $\times$ $O(\log^2 n) = O(kn^2 \log^2 n)$ (section 2.9).

3. Partie A — le glouton est presque optimal

2.1 Noyaux de visibilité et le cas d'un seul gardien (Obs. 2.1, 2.2 — Figure 2 du site)

> **Définition (noyau de visibilité).** Pour un ensemble E d'arêtes orientées de P, le *noyau de visibilité* P_E est l'ensemble des points du plan situés à gauche (ou sur) la droite support de chaque arête de E. C'est une intersection de demi-plans : un polygone convexe, éventuellement vide ou non borné.

> **Observation 2.1.** Si $(g, [u, v])$ est un gardien contigu, alors $g \in P_E$ où E est l'ensemble des arêtes rencontrées par la chaîne ouverte (u, v).

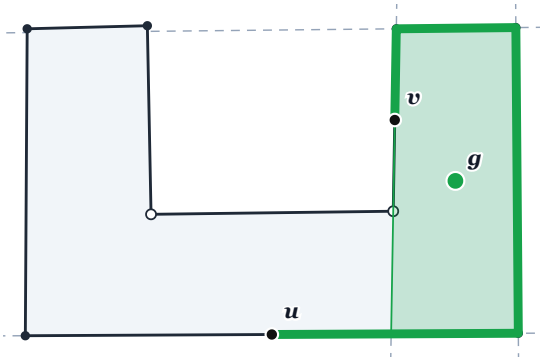
Preuve. Supposons g strictement à droite de la droite support d'une arête $e \in E$. Prenons un point q dans l'intérieur relatif de $e \cap (u, v)$. Juste avant d'atteindre q, le segment gq arrive du côté droit de e, c'est-à-dire de l'*extérieur* de P (l'intérieur est à gauche). Donc $gq \not\subseteq P$ et g ne voit pas q : contradiction. □

> **Observation 2.2.** Un seul gardien contigu suffit ($k = 1$) si et seulement si le noyau de toutes les arêtes de P (le *kernel*) est non vide, c'est-à-dire si P est étoilé. On le teste en temps linéaire (Lee–Preparata [13]).

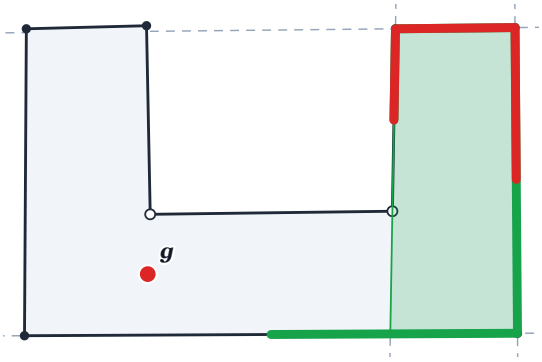
Preuve. ($\Rightarrow$) Observation 2.1 avec $E =$ toutes les arêtes. ($\Leftarrow$) Soit p dans le noyau et $q \in P$ quelconque. Partons de q et avançons en ligne droite vers p. Si le segment sortait de P, au point de sortie il traverserait une arête e de sa gauche (l'intérieur) vers sa droite (l'extérieur) ; le mouvement étant rectiligne, tous les points suivants, et en particu-

lier p, seraient strictement à droite de la droite support de e — impossible puisque p est à gauche de toutes les arêtes. Donc $qp \subseteq P$: p voit tout P, et en particulier tout le bord. □

Ce qu'il faut retenir. L'observation 2.2 est utilisée partout ensuite, mais appliquée à de *petits polygones* $Q \subseteq P$ construits dans la preuve : pour montrer qu'un point c voit une chaîne $[u, v]$, on trouve un polygone $Q \subseteq P$ dont le bord contient $[u, v]$ et on montre que c est dans son noyau.



Chaîne $[u, v]$ (vert), droites support de ses arêtes (pointillés) et noyau P_E (vert clair). Le point $g \in P_E$ voit toute la chaîne.



Un point hors de P_E ne voit pas toute la chaîne (parties rouges) : c'est l'observation 2.1.

2.2 L'algorithme glouton (preuve du théorème 7.1 — Figure 3)

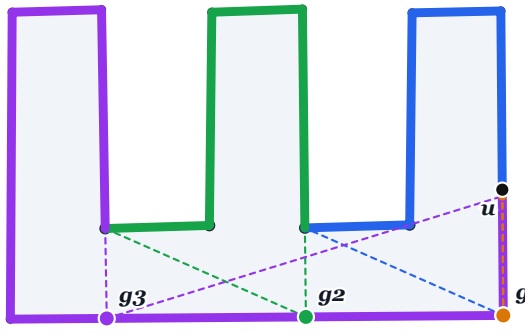
Pour $u \in \partial P$, notons $\text{next}(u)$ le point v le plus loin (dans le sens anti-horaire) tel qu'*un* gardien voie toute la chaîne $[u, v]$. Ce maximum existe car l'ensemble des couples (gardien, chaîne vue) est fermé. Le **glouton depuis** u prend le gardien qui réalise $\text{next}(u)$, repart de $\text{next}(u)$, et ainsi de suite jusqu'à ce que les chaînes recouvrent ∂P. On note G_u le résultat.

> **Lemme (glouton).** Pour tout u, $|G_u| \leq k + 1$. Si une solution optimale contient un gardien dont la chaîne *commence* en u, alors $|G_u| = k$.

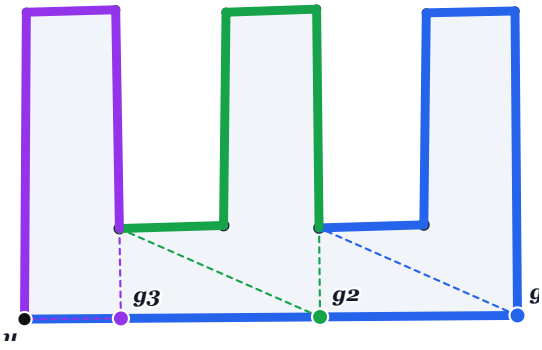
Preuve (« le glouton reste en avance »). Fixons une solution optimale G^* et numérotons ses chaînes $[u_0^*, v_0^*], [u_1^*, v_1^*], \ldots$ dans l'ordre du bord, en commençant par celle qui contient u (indice 0). Les chaînes du glouton sont $[u_1, v_1], [u_2, v_2], \ldots$ avec $u_1 = u$ et $u_{i+1} = v_i$. Montrons par récurrence que $v_i \geq v_{i-1}^*$ (positions mesurées le long du bord à partir de u).

- *Initialisation.* Le gardien optimal d'indice 0 voit $[u_0^*, v_0^*] \ni u$, donc il voit $[u, v_0^*]$; par définition de next, $v_1 = \text{next}(u) \geq v_0^*$.
- *Hérédité.* Supposons $v_i \geq v_{i-1}^*$. Si $u_{i+1} = v_i > v_i^*$, on a directement $v_{i+1} \geq u_{i+1} > v_i^*$. Sinon, comme les chaînes optimales se touchent ($u_i^* \leq v_{i-1}^*$), on a $u_{i+1} \in [u_i^*, v_i^*]$; le i-ème gardien optimal voit donc $[u_{i+1}, v_i^*]$, d'où $v_{i+1} = \text{next}(u_{i+1}) \geq v_i^*$.

Après $k + 1$ pas, $v_{k+1} \geq v_k^*$, qui est v_0^* après un tour complet, donc au-delà de u + un tour : tout le bord est couvert, d'où $|G_u| \leq k + 1$. Si u est le *début* d'une chaîne optimale, on numérote cette chaîne 1 au lieu de 0 et la même récurrence donne $v_i \geq v_i^*$: k pas suffisent. □



Glouton depuis un mauvais départ : 4 gardiens ($= k + 1$).



Glouton depuis le début d'une chaîne optimale : 3 gardiens ($= k$).

Conséquence. Pour tout u, $k \in \{|G_u|-1, |G_u|\}$. Tout le problème est de trouver un bon point de départ. Merrild et al. [2] relancent le glouton $O(kn^3)$ fois ; l'apport de [5] est un ensemble U_P de seulement $O(n^2)$ départs, dont l'un est garanti être le début d'une chaîne optimale.

Remarque (monotonie). next est croissante : si $x \le x'$ et qu'un gardien voit $[x, v]$, il voit aussi $[x', v]$. C'est pour cela que la comparaison « le glouton reste en avance » fonctionne.

4. Partie B — bons et mauvais gardiens

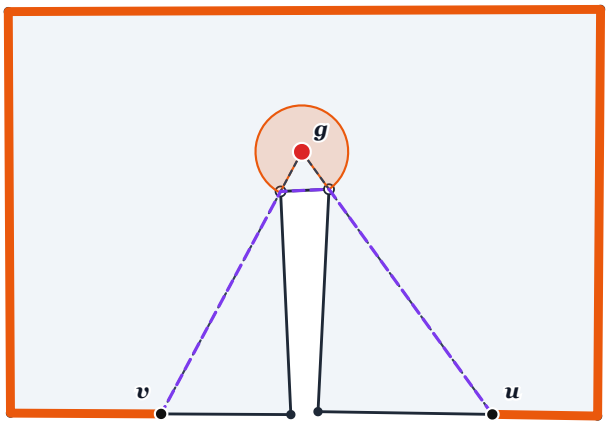
2.3 Définition (Déf. 4.1, Obs. 4.8 — Figure 4)

Définition 4.1. Un gardien contigu $(g, [u, v])$ est **mauvais** s'il vérifie les quatre conditions : (I) la chaîne ouverte (u, v) contient un sommet de P ; (II) $u \neq g$ et $v \neq g$; (III) l'angle $\angle(v, g, u) > \pi$; (IV) le plus court chemin de u à v dans P n'est pas un seul segment. Il est **bon** sinon.

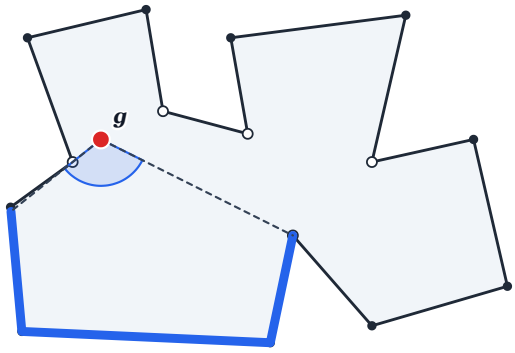
Intuition. Comme g est à gauche de toutes les arêtes de la chaîne (obs. 2.1), la chaîne tourne dans le sens anti-horaire autour de g ; l'angle de la condition (III) est l'angle balayé de la direction de u à celle de v. Un bon gardien regarde sa chaîne « de face » (angle $\le \pi$) ; un mauvais gardien regarde « autour d'un obstacle » : il voit plus d'un demi-tour de mur alors que u et v ne se voient pas. Les conditions (I), (II), (IV) excluent les cas dégénérés faciles (chaîne dans une seule arête, gardien sur le bord, u voit v).

Observation 4.8. Si g voit u et v et $\angle(v, g, u) > \pi$, le plus court chemin de u à v est une chaîne convexe qui tourne toujours à gauche.

Idée. Le chemin $u \to g \to v$ est dans P ; en le « tendant », on obtient le plus court chemin, qui reste dans le triangle ugv côté obstacle et s'enroule autour des sommets réflexes qui bloquent le segment uv. Comme l'angle en g dépasse π, ces sommets sont tous du même côté : la chaîne est convexe.



Un **mauvais** gardien dans la salle à encoche : angle $296° > \pi$, et le plus court chemin $u \to v$ (violet) contourne l'encoche.



Un **bon** gardien : il voit sa chaîne sous un angle de $115° \le \pi$.

2.4 Remplacer un gardien par un sommet de l'arrangement (Prop. 4.3, Lemmes 4.2–4.6, Cor. 4.7, Lemme 4.9 — Figure 5)

Soit C_P l'**arrangement** des n droites support des arêtes ; il a $O(n^2)$ sommets. Pour un gardien $(g, [u, v])$ on note E les arêtes rencontrées par (u, v), P_E leur noyau, et P_g le polygone délimité par le segment gu, la chaîne $[u, v]$ et le segment vg. Comme g voit u, v et toute la chaîne, $P_g \subseteq P$.

Proposition 4.3. Si $[u, v]$ contient un sommet de P, alors ou bien un sommet de P_E est dans P_g, ou bien un sommet de P appartenant à $[u, v]$ est dans P_E.

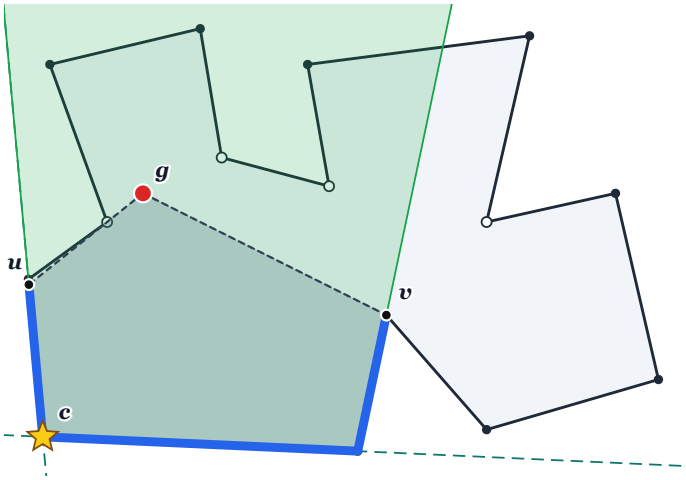
Preuve. Le bord de P_E est une chaîne convexe C. Comme $g \in P_E$ et que u et v sont sur les droites support de la première et de la dernière arête de E, la chaîne C traverse les segments gu et vg ; soit C' sa partie dans P_g. Si C' ne contient aucun sommet de P_E, c'est un morceau d'une seule arête de P_E, portée par la droite support d'une arête $e \in E$, avec ses deux extrémités hors de P_g. Ce segment coupe P_g en traversant la chaîne là où se trouve e : une extrémité de e appartient à $[u, v]$ et se trouve sur C', donc dans P_E. □

Le point fourni par la proposition est dans P_E (à gauche de toutes les arêtes de la chaîne). S'il est *aussi* à gauche des segments gu et vg, il est dans le noyau de P_g et voit donc toute la chaîne par l'observation 2.2. C'est exactement ce que fournit chaque condition de la définition 4.1 qui échoue :

- **Lemme 4.2 (I échoue).** (u, v) est contenue dans une seule arête ab : le sommet a voit toute l'arête (segment contenu dans P).
- **Lemme 4.4 (II échoue, $g = u$).** P_g est délimité par $[u, v]$ et le segment vu ; tout P_g est à gauche de vu, donc le point trouvé est dans le noyau de P_g.
- **Lemme 4.5 (III échoue).** $\angle(v, g, u) \leq \pi$: P_g est entièrement à gauche de gu et de vg (c'est le sens de « angle $\leq \pi$ »), même conclusion.
- **Lemme 4.6 (IV échoue, u voit v).** On travaille dans P' délimité par $[u, v]$ et vu : $P_g \subseteq P' \subseteq P$ et le noyau de P' vaut $P_E \cap P'$. Le point trouvé (dans $P_g \cap P_E$) est donc dans le noyau de P' et voit $[u, v]$.

> **Corollaire 4.7.** Pour tout bon gardien $(g, [u, v])$, il existe un gardien $(c, [u, v])$ où c est un sommet de C_P, intersection des droites support de deux arêtes e_1, e_2. Sauf si $[u, v]$ ne contient aucun sommet de P, on peut choisir c tel que $[u, v]$ contienne une extrémité de e_1 et une de e_2.

(Un sommet de P_E est l'intersection de deux droites support d'arêtes de E, qui rencontrent (u, v) ; un sommet de P est l'intersection des droites de ses deux arêtes.)

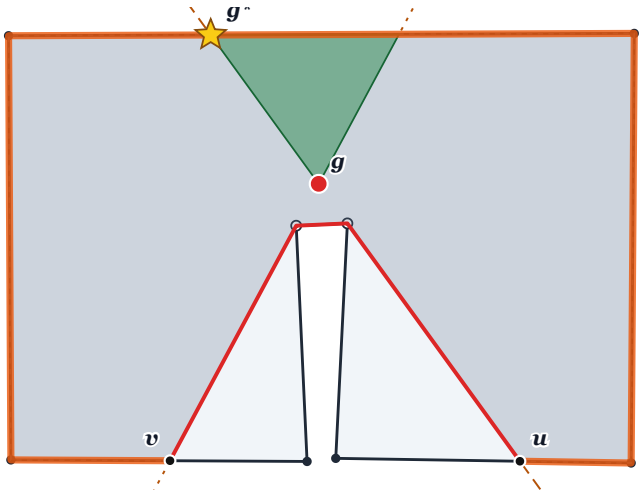


Bon gardien (lemme 4.5) : P_g en gris, P_E en vert. L'étoile c est un sommet de P_E dans P_g, intersection de deux droites support (pointillés) : elle voit la même chaîne.

Pourquoi les mauvais gardiens posent problème. Si l'angle dépasse π, le polygone P_g n'est plus à gauche de gu et vg : un sommet de P_E dans P_g peut être caché de u par l'obstacle autour duquel le gardien regarde. L'article décrit quand même où trouver un remplaçant, mais il dépend de u :

> **Lemme 4.9.** Soit $(g, [u, v])$ mauvais, v sur l'arête $v_j v_{j+1}$, et ℓ_u la droite support de la dernière arête du plus court chemin de v_{j+1} à u. Alors un sommet de P_E, ou le premier ou le dernier point de ℓ_u dans P_E, voit aussi $[u, v]$.

Preuve. Soit S_v le plus court chemin de v à u ; par l'observation 4.8 il est convexe, donc son noyau est l'intersection des demi-plans à gauche des droites ℓ_v et ℓ_u portant sa première et sa dernière arête. Le polygone P_v délimité par S_v et $[u, v]$ a pour noyau $P' = P_E \cap \{\text{gauche de } \ell_u, \ell_v\}$, qui contient g. Si une extrémité de $\ell_u \cap P_E$ est dans P', elle est dans le noyau de P_v et voit $[u, v]$. Sinon P' est borné et non vide, et l'un de ses sommets est un sommet de P_E (ou de P), qui voit $[u, v]$ pour la même raison. □



Lemme 4.9 sur le mauvais gardien de la figure précédente : P_v (gris), P' (vert foncé), S_v (rouge), ℓ_u et ℓ_v (orange). Le point g^* où ℓ_u quitte P_E voit toute la chaîne.

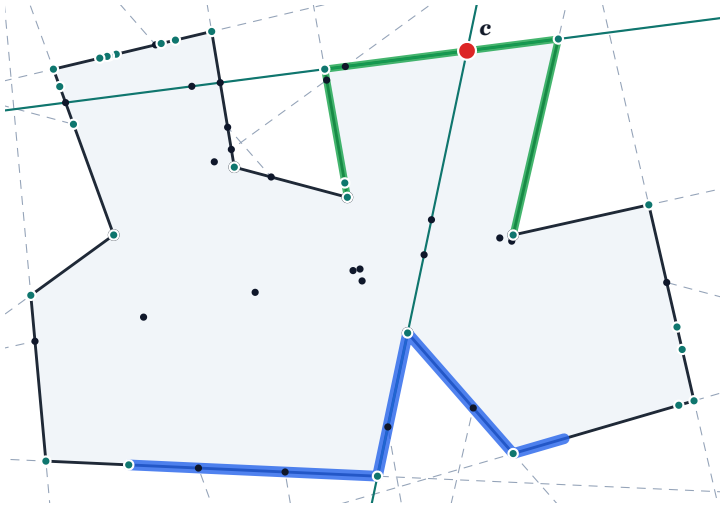
5. Partie C — discrétisation

2.5 Gardiens candidats G_P et points de départ U_P (Déf. 5.1, 5.2, Lemme 6.6 — Figure 6)

Définition 5.1. Pour chaque sommet c de C_P, défini par les arêtes ab et ef, et chaque $x \in \{a, b, e, f\}$ vu par c, soit $[u_x, v_x]$ l'unique chaîne maximale (pour l'inclusion) vue par c qui contient x. G_P est l'ensemble de ces gardiens $(c, [u_x, v_x])$.

Définition 5.2. U_P = tous les sommets de P et tous les débuts u_x des gardiens de G_P. On a $|G_P|, |U_P| = O(n^2)$.

« Unique » : un point fixe voit un ensemble fixe de chaînes maximales disjointes (sa région de visibilité coupe le bord en morceaux), donc il y en a une seule qui contient x.



Polygone « galerie » ($n = 15$) : droites support, 42 sommets de C_P dans P, $|G_P| = 49$, $|U_P| = 29$ (points verts sur le bord). Le sommet c (rouge) et ses chaînes candidates.

Calcul (lemme 6.6, $O(n^2 \log n)$). On parcourt l'arrangement le long d'une marche qui passe par tous ses sommets. Pour le sommet courant on maintient, dans un arbre équilibré, la chaîne de bits « suis-je à gauche de la droite i ? » ; entre deux sommets voisins elle change en au plus deux positions. En chaque sommet c et pour chaque extrémité x : une requête de plus court chemin teste si c voit x ; l'arbre donne la plus longue suite d'arêtes ayant c à gauche ($E_{c,x}$, déf. 6.1) ; le lemme 6.5 donne $[u_x, v_x]$ en $O(\log n)$.

2.6 Théorème 5.4 : une solution optimale commence dans U_P (Lemme 5.3 — Figure 7)

> **Lemme 5.3.** Si $(g, [u, v])$ est un gardien avec $[u, v] \neq \partial P$ maximale, alors ou bien v est un sommet réflexe, ou bien le segment vg contient un sommet réflexe en son intérieur.

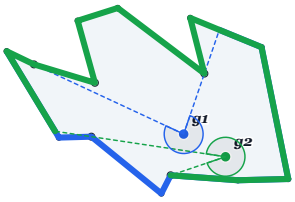
Preuve. Juste après v le bord n'est plus vu par g (maximalité). Pour $\varepsilon > 0$ petit, le plus court chemin de g à $v + \varepsilon$ se plie donc en un sommet réflexe x. Quand $\varepsilon \to 0$, ce chemin tend vers le segment gv : soit $x = v$, soit x est à l'intérieur du segment vg. □

> **Théorème 5.4.** Si G est une solution minimale avec $|G| > 1$, il existe une solution G' de même taille dont un gardien $(g, [u, v])$ vérifie $u \in U_P$.

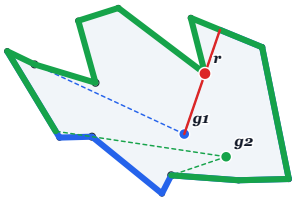
Preuve. On commence par étendre chaque chaîne à la chaîne maximale vue par son gardien.

Cas 1 : un gardien est bon. Si sa chaîne ne contient aucun sommet, le lemme 4.2 le remplace par un sommet de P dont la chaîne commence en un sommet ($\in U_P$). Sinon le corollaire 4.7 donne un sommet c de C_P qui voit la même chaîne $[u, v]$, avec $[u, v]$ contenant une extrémité x d'une arête définissant c. Le candidat $(c, [u_x, v_x]) \in G_P$ voit la chaîne maximale qui contient x, donc $[u, v] \subseteq [u_x, v_x]$: on remplace le gardien par ce candidat, sa chaîne commence en $u_x \in U_P$.

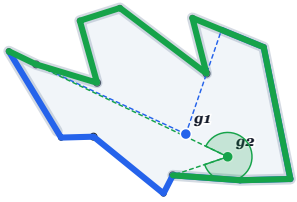
Cas 2 : tous les gardiens sont mauvais. Prenons $(g, [u, v])$. Si u est un sommet, c'est fini. Si v est un sommet, le gardien suivant (dont la chaîne contient v) peut être raccourci pour commencer en v. Sinon, par le lemme 5.3, vg passe par un sommet réflexe $r \notin [u, v]$; un autre gardien $(g', [u', v'])$ voit r. Si r est une extrémité de sa chaîne, on conclut comme ci-dessus. Sinon r est à l'intérieur de sa chaîne : g' est à gauche des deux arêtes en r, donc à gauche de la droite vg, et comme $\angle(v, g, u) > \pi$, cela force g' dans P_g. On retire de sa chaîne la partie (u, v) déjà couverte par g : il reste une seule chaîne $[a, b]$ (sinon G ne serait pas minimale), vue depuis P_g sous un angle $\angle(a, g', b) \leq \pi$. Le gardien raccourci est **bon** : on revient au cas 1. □



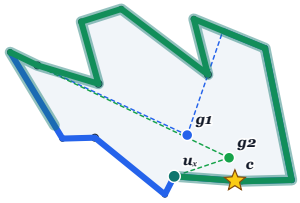
(1) Chaînes maximales : les deux gardiens sont **mauvais** (angles > π).



(2) Lemme 5.3 : le segment vg_1 passe par le sommet réflexe r.



(3) g_2 est raccourci : l'angle devient $\leq \pi$, il est **bon**.



(4) Corollaire 4.7 : g_2 remplacé par un candidat de G_P ; sa chaîne commence en $u_x \in U_P$.

6. Partie D — un pas glouton rapide

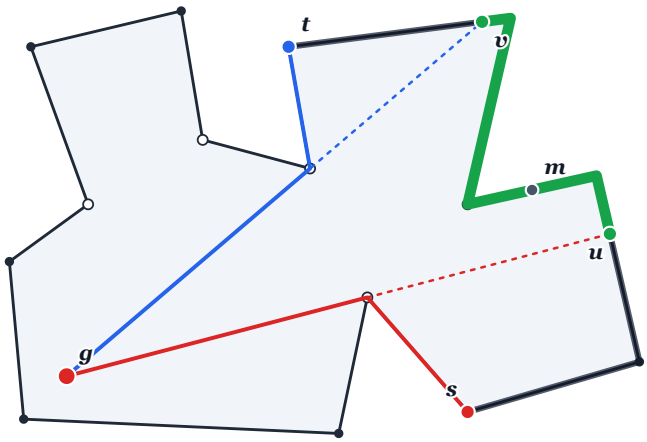
2.7 La chaîne visible maximale (Déf. 6.1, Lemmes 6.2, 6.5 — Figure 8)

> **Définition 6.1.** Pour $g \in P$ et $m \in \partial P$ vu par g, $E_{g,m}$ est la plus longue suite contiguë d'arêtes contenant m telle que g soit à gauche de la droite support de chacune.

> **Lemme 6.2.** Soient s, t le premier et le dernier sommet d'une sous-suite $E \subseteq E_{g,m}$ avec $m \in [s, t] \neq \partial P$. Soient π_s, π_t les plus courts chemins de s et t à g, et r_s, r_t les rayons partant de g le long de leur dernière arête. Ils touchent $[s, t]$ en u et v, et $[u, v]$ est l'unique chaîne maximale de $[s, t]$ vue par g qui contient m.

Preuve. Le segment gm coupe la région bordée par π_s, $[s, t]$, π_t en une partie P_s (côté s) et une partie P_t. Soit u^* le dernier point de $[s, m]$ tel que gu^* touche π_s en plus d'un point. Le polygone délimité par $[u^*, m]$, mg et gu^* ne contient aucun point de π_s ; il est donc dans P, et g est à gauche de toutes ses arêtes : g voit $[u^*, m]$ (obs. 2.2). Les points avant u^* sont cachés par π_s. Enfin gu^* longe la dernière arête de π_s (sinon on pourrait raccourcir π_s) : u^* est le point où r_s touche la chaîne. Idem pour v. □

Notre remarque d'implémentation. Quand la chaîne $[s,t]$ fait plus d'un tour autour de g, le rayon r_t peut toucher $[s,t]$ *deux fois* (par exemple quand π_t passe par s). L'énoncé « les rayons touchent $[s,t]$ en u et v » devient ambigu ; c'est la définition de la preuve qui tranche : u est le point de $[s,m]$ le plus proche de m, et v celui de $[m,t]$ le plus proche de m. Notre première version prenait le point le plus proche de g et se trompait dans environ 19 % des cas aléatoires ; avec la bonne règle, elle coïncide avec un calcul direct de visibilité sur tous nos tests.



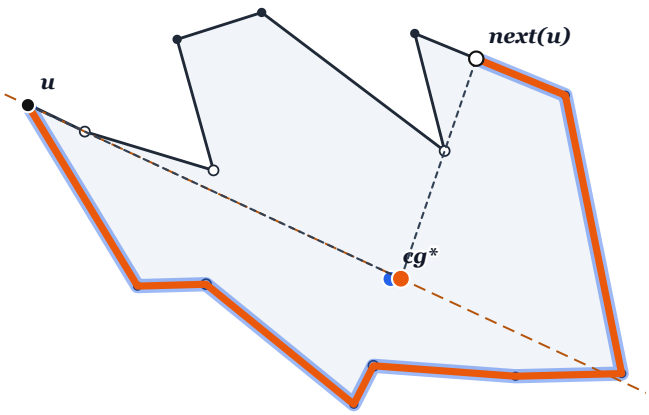
Lemme 6.2 : chaîne E de s à t (noir), plus courts chemins π_s (rouge) et π_t (bleu), rayons, et le résultat $[u,v]$ (vert).

Structures de données. Guibas–Hershberger [11] : après $O(n)$ prétraitement, le plus court chemin entre deux points est rendu en $O(\log n)$ sous forme de $O(\log n)$ arbres équilibrés, ce qui donne sa dernière arête en $O(\log n)$. Hershberger–Suri [14] : le premier point du bord touché par un rayon en $O(\log n)$. Donc le lemme 6.5 coûte $O(\log n)$.

2.8 Un pas glouton en $O(\log^2 n)$ (Lemme 6.8 — Figure 9)

La structure de données comporte : (1) les structures du lemme 6.2 ; (2) les noyaux de toutes les suites de 2^j arêtes commençant ou finissant à chaque arête, pour obtenir le noyau de n'importe quel intervalle d'arêtes comme intersection de deux noyaux stockés ; (3) les chaînes de G_P triées par début, sans les chaînes dominées.

Requête $\text{next}(u)$**.** (a) *Bons gardiens* : chercher la chaîne candidate $[u',v']$ contenant u qui va le plus loin (recherche binaire) ; par le corollaire 4.7, si le gardien optimal est bon, $v' \geq \text{next}(u)$. (b) *Mauvais gardiens* : recherche binaire sur l'arête e où pourrait finir un mauvais gardien. Pour une arête e : le plus court chemin de l'extrémité de e à u doit être convexe (obs. 4.8) ; si oui, on calcule ℓ_u, on l'intersecte avec P_E (deux noyaux stockés), et on applique le lemme 6.5 au premier et au dernier point de $\ell_u \cap P_E$. Le lemme 4.9 garantit qu'on trouve ainsi le mauvais gardien (ou qu'un candidat faisait déjà aussi bien). $O(\log n)$ étapes de recherche binaire à $O(\log n)$ chacune : $O(\log^2 n)$.



Polygone « deux mauvais gardiens » : le meilleur candidat de G_P (bleu, gardien c) atteint 7.54 arêtes après u ; le mauvais gardien g^* trouvé sur ℓ_u atteint 7.58. C'est lui qui réalise $\text{next}(u)$.

7. L'algorithme complet et l'algorithme optimal

2.9 Théorème 7.1 (Figure 10)

> **Théorème 7.1.** On calcule une solution optimale en $O(kn^2 \log^2 n)$ temps et $O(n^2)$ espace.

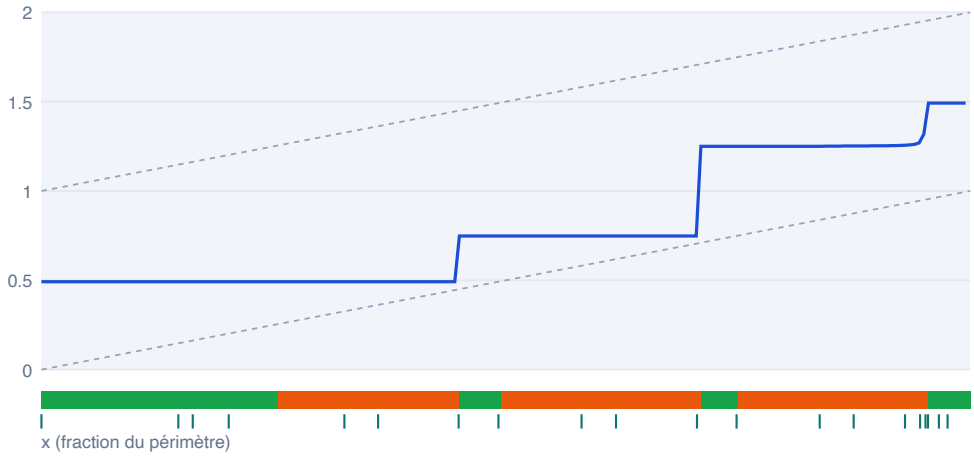
1. Si le noyau de P est non vide : un gardien suffit (obs. 2.2), $O(n)$.
2. Calculer U_P et la structure du lemme 6.8 : $O(n^2 \log n)$.
3. Pour chacun des $O(n^2)$ départs $u \in U_P$, lancer le glouton : au plus $k+1$ pas de $O(\log^2 n)$.
4. Rendre la plus petite solution. Par le théorème 5.4, un $u \in U_P$ est le début d'une chaîne optimale, et le glouton depuis ce u rend k gardiens.

Total : $O(n^2 \log n) + O(n^2) \cdot O(k) \cdot O(\log^2 n) = O(kn^2 \log^2 n)$. Gain par rapport à $O(kn^5 \log n)$: un facteur $\Theta(n^3 / \log n)$.

> **Observation expérimentale.** Sur 373 polygones aléatoires non étoilés, en médiane 47 % des points de U_P donnent déjà une solution optimale (au minimum 4,7 %). Dans ces tests, partir des seuls sommets de P suffisait toujours — mais rien ne le garantit en général : la preuve a besoin des points u_x, et l'algorithme optimal de [6] utilise lui aussi « sommets + débuts de dominateurs ».

2.10 Vers $\Theta(n \log n)$ ([6] — Figure 11)

- **Ensemble de départs linéaire.** Tout bon gardien est dominé par un *dominateur bon* (au plus un par paire d'indices (i, j) : « segment-intersecting » si le noyau $E[i-1, j+1]$ coupe le segment $v_i v_j$, sinon « angle-maximizing »). En ne gardant que les dominateurs non strictement dominés, il en reste $O(n)$, qui définissent un ensemble X de $O(n)$ départs. Les mauvais gardiens sont dominés par des dominateurs *mauvais* ($O(n)$ après réduction) ou *moches* (« ugly »), calculés à la volée.
- **Fenêtre glissante.** Les noyaux $E[i-1, j+1]$ sont maintenus quand i et j ne font qu'augmenter ; par dualité c'est une enveloppe convexe sous des mises à jour « premier entré, premier sorti » (Chan–Hershberger–Pratt). On calcule $\mathrm{next}(x)$ pour tout $x \in X$ en $O(n \log n)$ au total, d'où $O(kn \log n)$.
- **Supprimer k.** next est une fonction par morceaux à $O(n)$ morceaux, chacun constant ou une transformation de Möbius $x \mapsto \frac{A+Bx}{C+Dx}$. Après une révolution de taille k', on sait que $k \in \{k'-1, k'\}$; une évaluation paresseuse dans un arbre équilibré applique next à tout X simultanément et teste en $O(n \log n)$ si un x fait le tour avec $k'-1$ gardiens.
- **Borne inférieure.** Réduction depuis *set disjointness* ($\Omega(n \log n)$ dans le modèle real RAM) : on construit un polygone dont une longue arête a une « indentation » pour chaque $a \in A$ et une « poche » pour chaque $b \in B$, gardable seulement depuis le point correspondant à b ; il est gardable avec $2|B|+2$ gardiens ssi $A \cap B = \emptyset$.



Graphe de $x \mapsto \mathrm{next}(x)$ (peigne, $k = 3$), axes en fraction du périmètre ; pointillés : $y = x$ et $y = x + 1$. Les paliers viennent de gardiens fixes, les parties courbes de gardiens qui bougent avec x. Bande : taille du glouton depuis x (vert $= k$, orange $= k + 1$) ; traits : U_P, qui tombent bien dans les zones vertes.

8. Répartition proposée des explications de la partie 2

Suivant la remarque du professeur, nous travaillons tous les deux sur la partie 2 et nous répartissons les explications. Proposition (modifiable dans `src/sections/blocks.jsx` et dans les attributs `author` de `src/sections/Part2.jsx`) :

Section	Contenu	Explique
1	Le problème (bref)	Ilias
2.1, 2.2	Noyaux, glouton, borne $k + 1$	Ilias
2.3, 2.4	Bons/mauvais gardiens, lemmes de remplacement, lemme 4.9	Ilias
2.5, 2.6	G_P, U_P, théorème 5.4	Jomaa
2.7, 2.8	Lemme 6.2, structure de données du lemme 6.8	Jomaa
2.9	Algorithme complet, complexité	Ilias
2.10	Algorithme optimal $\Theta(n \log n)$	Jomaa
3, 4	Jeu, bac à sable, rapport	les deux

Glossaire français – anglais

gardien contigu	contiguous guard	chaîne (du bord)	chain
noyau de visibilité	visibility core	noyau (kernel), polygone étoilé	kernel, star-shaped
droite support	supporting line	arrangement de droites	line arrangement
sommet réflexe	reflex vertex	plus court chemin	shortest path
lancer de rayon	ray shooting	révolution (du glouton)	revolution

Références

1. Biniaz, Maheshwari, Merrild, Mitchell, Odak, Polishchuk, Robson, Rysgaard, Schou, Shermer, Spalding-Jamieson, Svenning, Zheng. Polynomial-time algorithms for contiguous art gallery and related problems. SoCG 2025.
2. Merrild, Rysgaard, Schou, Svenning. The contiguous art gallery problem is solvable in polynomial time. arXiv:2412.13938.
3. Biniaz, Bose, Maheshwari, Mitchell, Odak, Polishchuk, Shermer. Contiguous boundary guarding. arXiv:2412.15053.
4. Robson, Spalding-Jamieson, Zheng. The analytic arc cover problem… arXiv:2412.15567.
5. **de Berg, Conradi, van der Hoog, Staals. Simpler and faster contiguous art gallery. arXiv:2508.09734, 2025.**
6. de Berg, Conradi, van der Hoog, Rotenberg. The contiguous art gallery problem is in Θ(n log n). arXiv:2511.02960.
7. Chvátal. A combinatorial theorem in plane geometry. JCTB 1975.
8. Lee, Lin. Computational complexity of art gallery problems. IEEE Trans. Inf. Theory 1986.

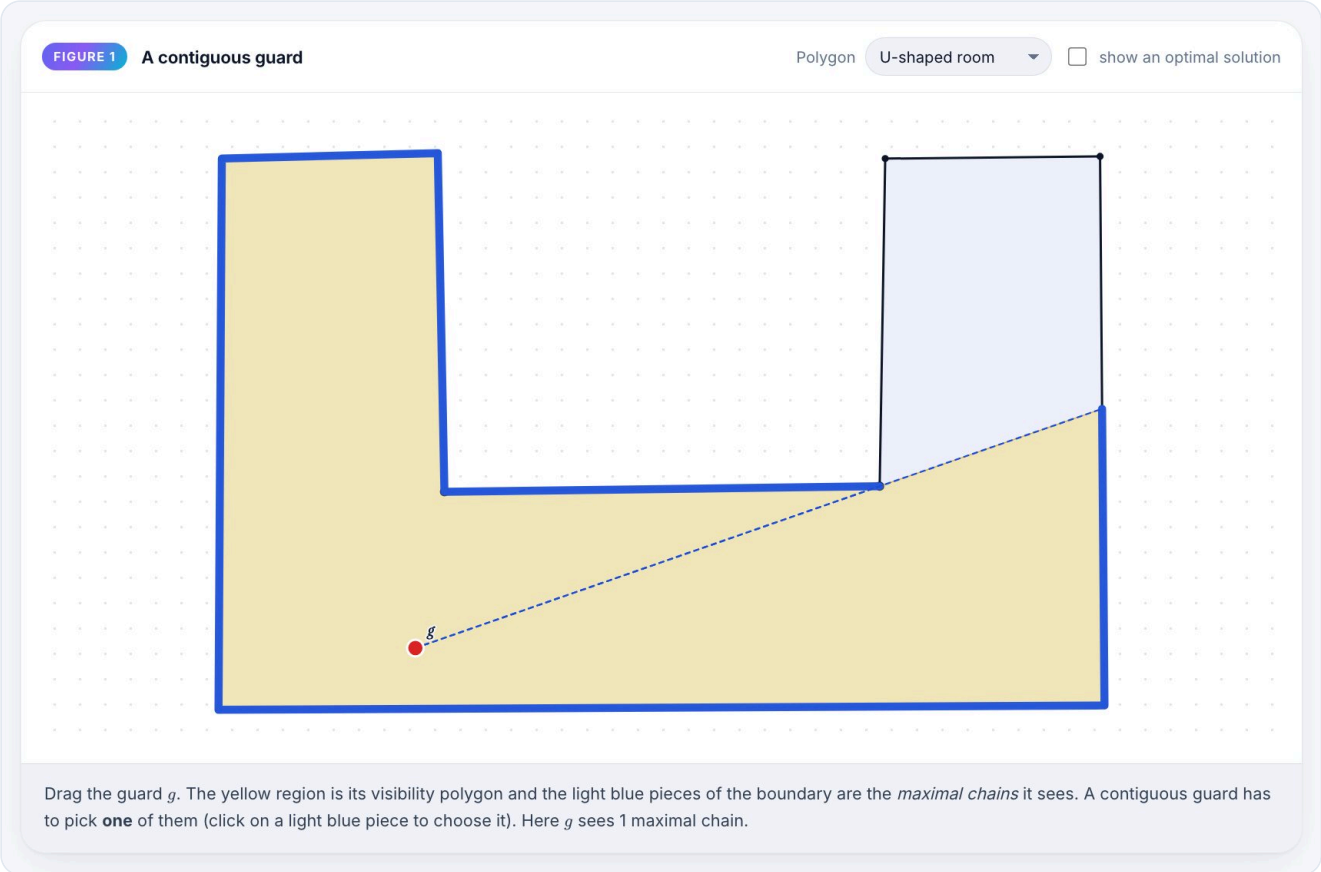
9. Abrahamsen, Adamaszek, Miltzow. The art gallery problem is $\exists\mathbb{R}$-complete. J. ACM 2022.
10. Stade. The point-boundary art gallery problem is $\exists\mathbb{R}$-hard. SoCG 2025.
11. Guibas, Hershberger. Optimal shortest path queries in a simple polygon. JCSS 1989.
12. Guibas, Hershberger, Leven, Sharir, Tarjan. Linear-time algorithms for visibility and shortest path problems… Algorithmica 1987.
13. Lee, Preparata. An optimal algorithm for finding the kernel of a polygon. J. ACM 1979.
14. Hershberger, Suri. A pedestrian approach to ray shooting. SODA 1993.

E

Les figures interactives

Où les trouver, ce qu'elles montrent, comment s'en servir.

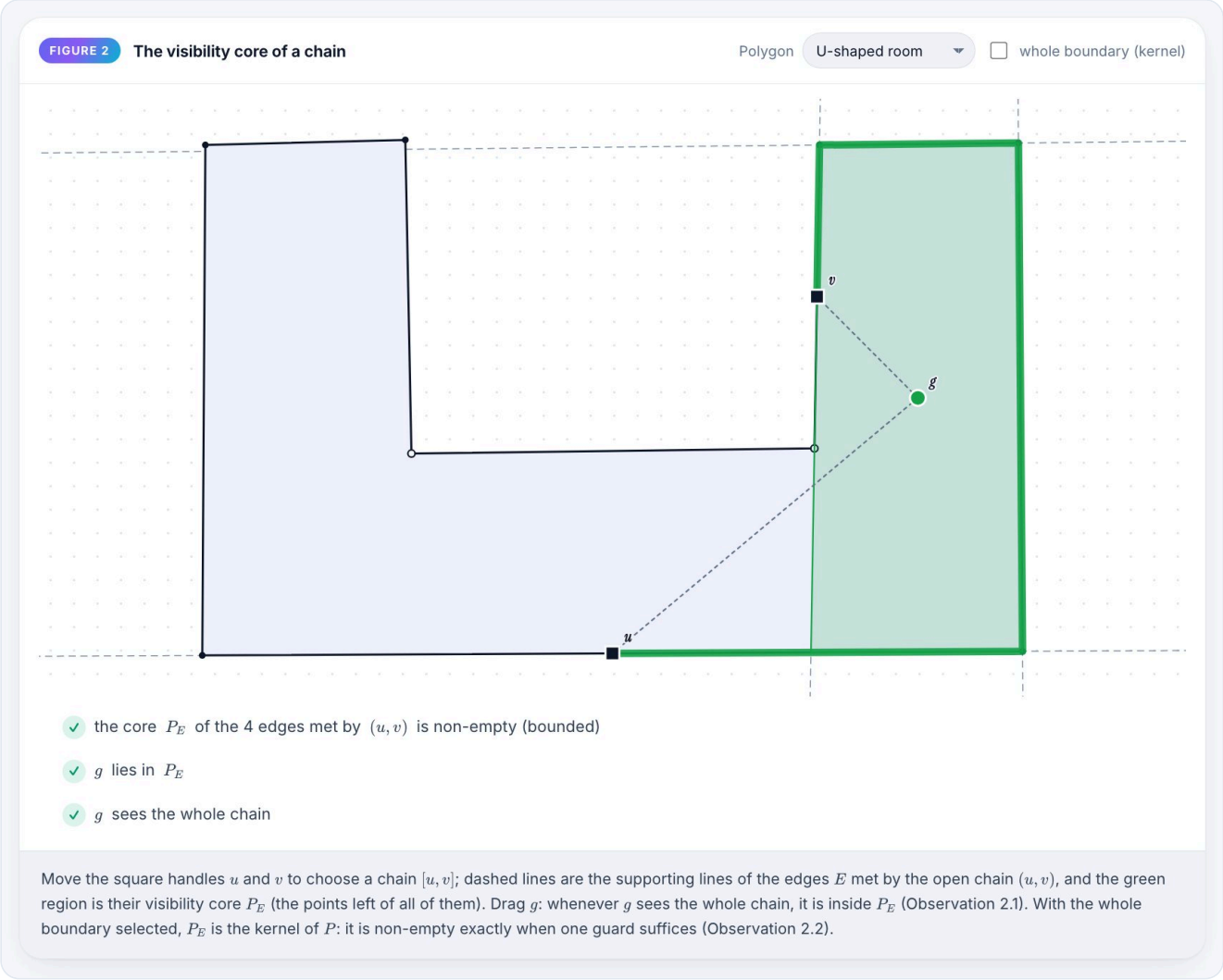
Figure 1 — Un gardien contigu Accueil, section 01



Ce qu'elle montre. La région jaune est ce que voit le point g ; les morceaux bleu clair sont les chaînes maximales qu'il voit. Un gardien contigu n'en garde qu'une.

Comment s'en servir. Déplacer g ; cliquer sur un morceau bleu clair pour le choisir ; cocher « show an optimal solution » pour voir les k gardiens optimaux.

Figure 2 — Le noyau de visibilité Chapitre 1, §2.1



Ce qu'elle montre. Les droites support des arêtes de la chaîne $[u, v]$ et leur noyau P_E (vert). Observation 2.1 : un gardien qui voit toute la chaîne est dans P_E.

Comment s'en servir. Déplacer les poignées carrées u, v et le point g ; cocher « whole boundary » : le noyau de P est vide ssi $k \geq 2$.

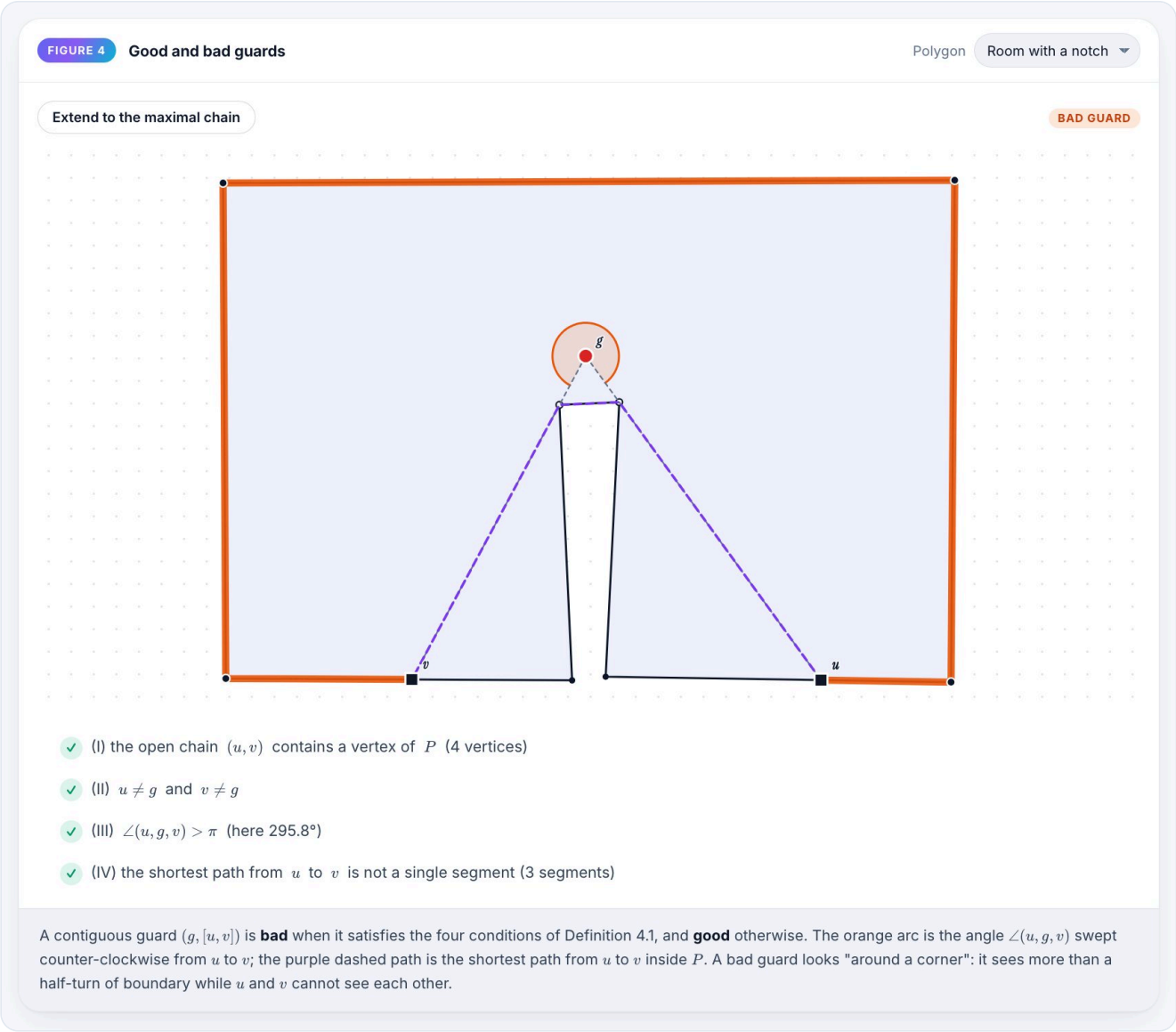
Figure 3 — L'algorithme glouton Chapitre 1, §2.2



Ce qu'elle montre. Le glouton pas à pas, et la bande « déroulée » qui compare ses chaînes à celles d'une solution optimale : il reste toujours en avance.

Comment s'en servir. « Step », « Run to the end », « Reset » ; déplacer u ; « Start at an optimal chain » montre que k gardiens suffisent depuis le bon départ.

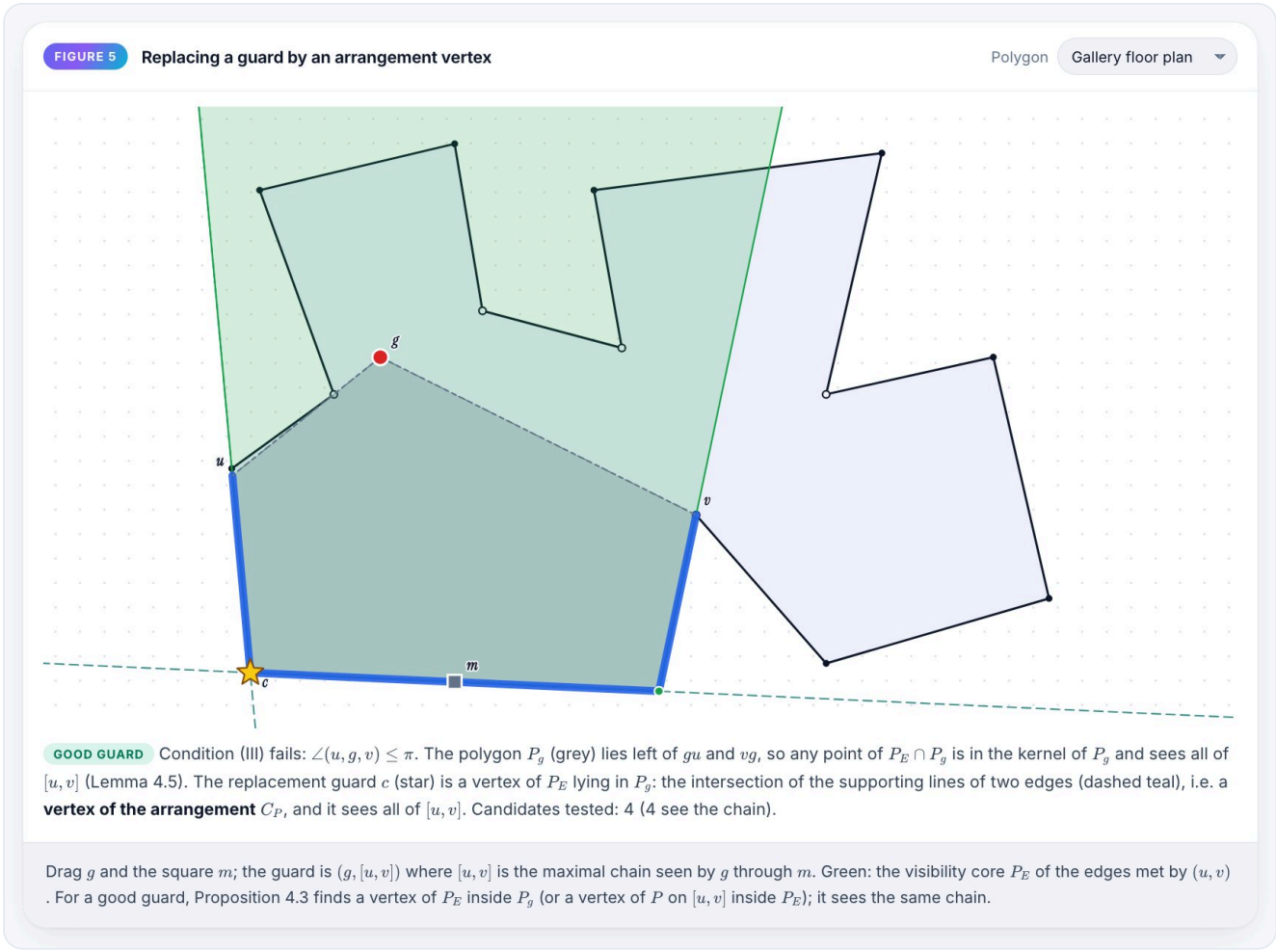
Figure 4 — Bons et mauvais gardiens Chapitre 2, §2.3



Ce qu'elle montre. Les quatre conditions de la définition 4.1 évaluées en direct, l'angle balayé (orange) et le plus court chemin de u à v (violet).

Comment s'en servir. Déplacer g, u, v ; « Extend to the maximal chain ». Sur la salle à encoche, le gardien par défaut est mauvais (296°).

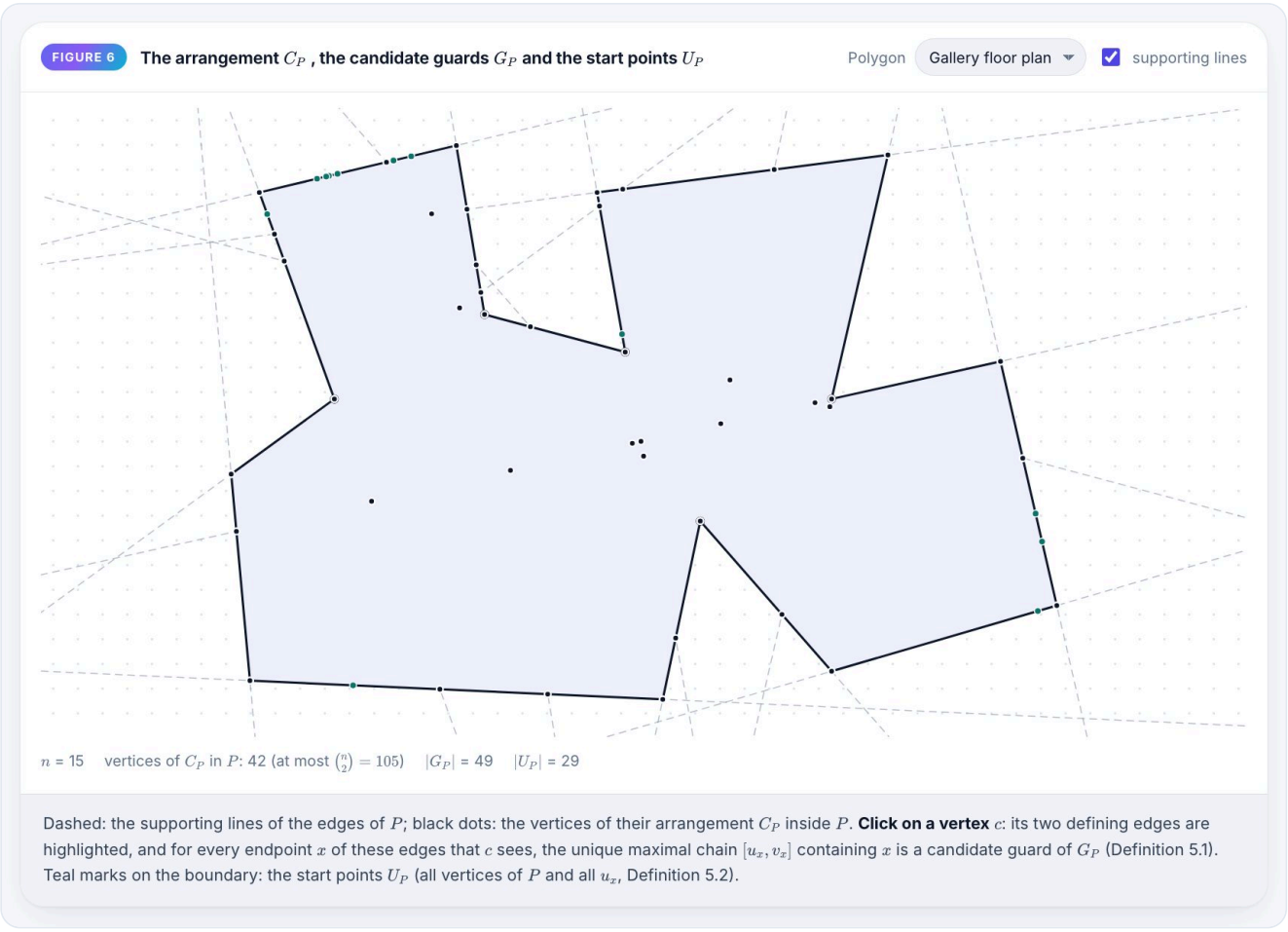
Figure 5 — Remplacer un gardien Chapitre 2, §2.4



Ce qu'elle montre. Bon gardien : P_g (gris), P_E (vert) et le remplaçant c (étoile), sommet de l'arrangement. Mauvais gardien : la construction du lemme 4.9 (P_v, P', ℓ_u, ℓ_v, g^*).

Comment s'en servir. Déplacer g et m ; choisir « Room with a notch » ou « Two bad guards » pour voir le cas mauvais.

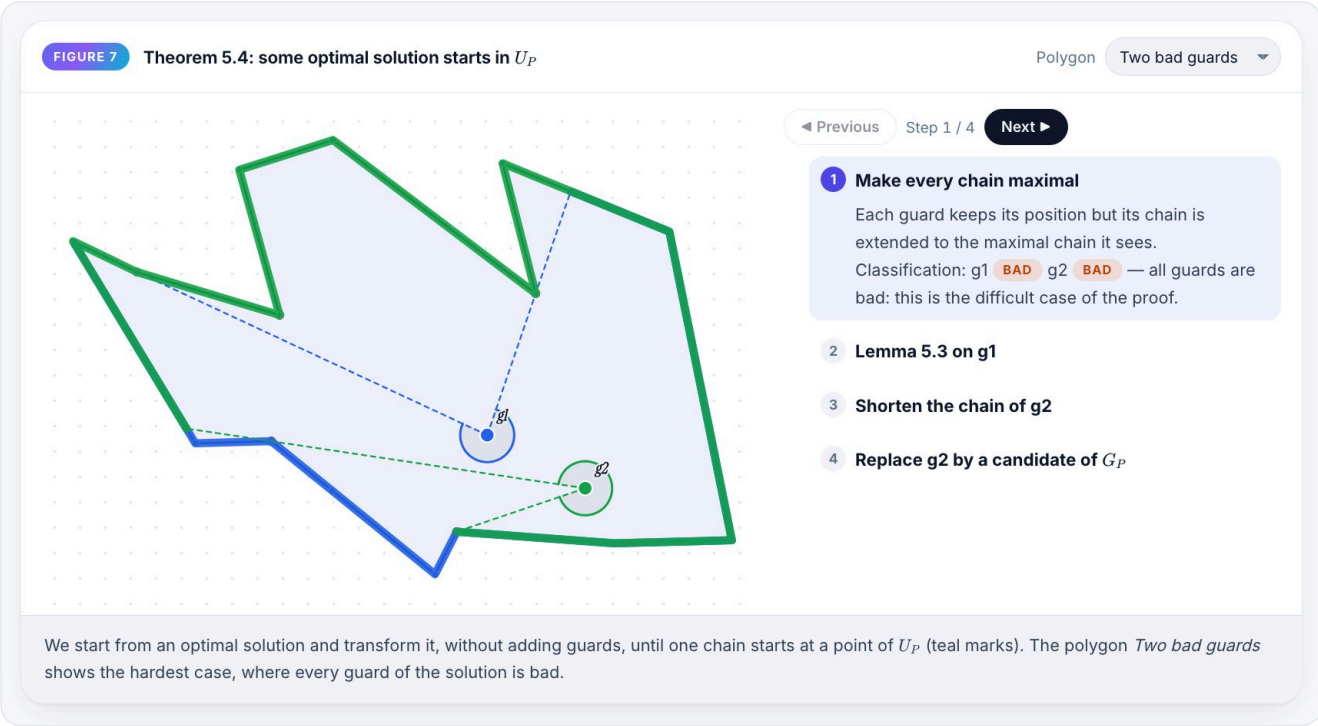
Figure 6 — Arrangement, G_P et U_P Chapitre 3, §2.5



Ce qu'elle montre. Toutes les droites support, les sommets de l'arrangement dans P et les points de départ U_P (marques vertes).

Comment s'en servir. Cliquer sur un sommet c : ses deux arêtes définissantes et ses chaînes candidates s'affichent.

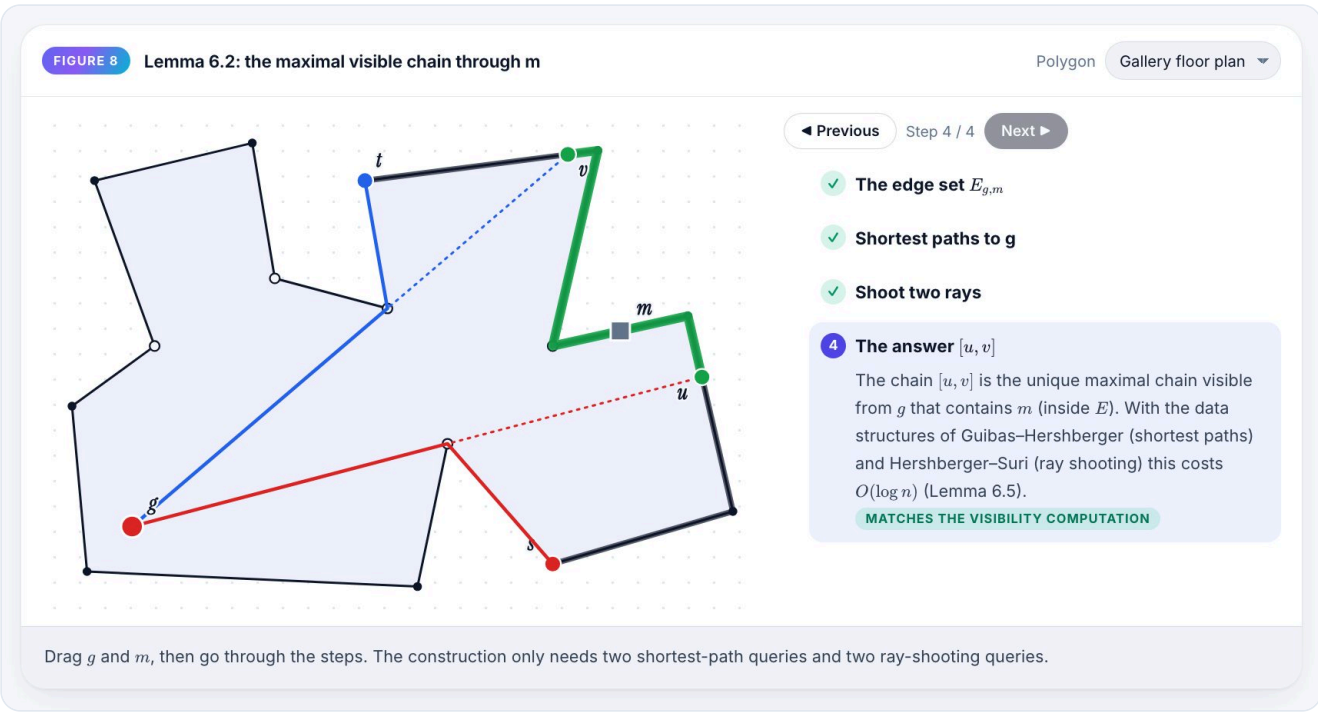
Figure 7 — Théorème 5.4 pas à pas Chapitre 3, §2.6



Ce qu'elle montre. La preuve appliquée à une vraie solution optimale faite de deux mauvais gardiens : chaînes maximales, lemme 5.3, raccourcissement, remplacement par un candidat de G_P.

Comment s'en servir. « Next » / « Previous », ou cliquer une étape.

Figure 8 — Lemme 6.2 Chapitre 4, §2.7



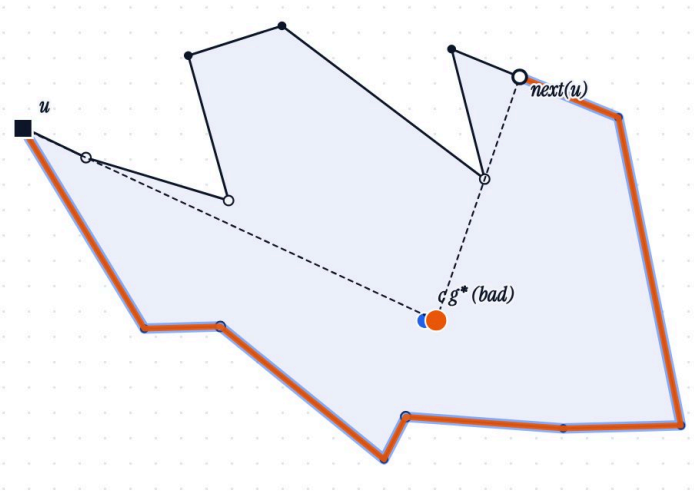
Ce qu'elle montre. $E_{g,m}$, les plus courts chemins π_s (rouge) et π_t (bleu), les deux rayons, et la chaîne maximale $[u, v]$ (vert), comparée à un calcul direct.

Comment s'en servir. Déplacer g et m puis dérouler les 4 étapes ; le badge vert confirme l'accord avec la visibilité calculée directement.

Figure 9 — Un pas glouton : $\mathrm{next}(u)$ Chapitre 4, §2.8

FIGURE 9 **One greedy step: computing $\mathrm{next}(u)$**

Polygon: Two bad guards



Good part: v' = 7.54 edges after u.
Result: $\mathrm{next}(u)$ is 7.58 edges after u, realised by a BAD GUARD FOUND ON A LINE ℓU

EDGE	PATH LEFT-TURNING?	POINTS ON ℓU	REACH
e2	✓	4	–
e3	✓	4	7.07
e4	✓	4	7.10
e5	✓	4	7.18
e6	✓	4	7.38
e7	✓	4	7.58
e8	✓	4	7.58

The paper performs a binary search over the edges (the predicate is monotone), giving $O(\log n)$ rounds of $O(\log n)$ each. For clarity we list every edge.

Lemma 6.8 computes $\mathrm{next}(u)$ in two parts. **Good guards:** look up the candidate chain of G_P containing u that reaches farthest (blue). **Bad guards:** for an edge e_j, take the shortest path from u to the end of e_j; if it turns only left, the line ℓ_u through its first edge is intersected with the visibility core P_E of the edges from u to e_j, and the first and last points of $\ell_u \cap P_E$ are tried as guards (Lemma 4.9). Click a row of the table to see that edge.

Ce qu'elle montre. La partie « bons gardiens » (meilleur candidat de G_P, en bleu) et la recherche des mauvais gardiens arête par arête (table).

Comment s'en servir. Déplacer u ; cliquer une ligne du tableau pour voir le chemin, ℓ_u et le noyau de cette arête.

Figure 10 — L'algorithme complet Chapitre 4, §2.9



Ce qu'elle montre. Chaque point de U_P coloré par la taille du glouton qui en part (vert $= k$, orange $= k+1$) et la meilleure solution.

Comment s'en servir. Cliquer une marque pour voir son glouton ; « Random polygon » et le curseur n pour tester d'autres galeries.

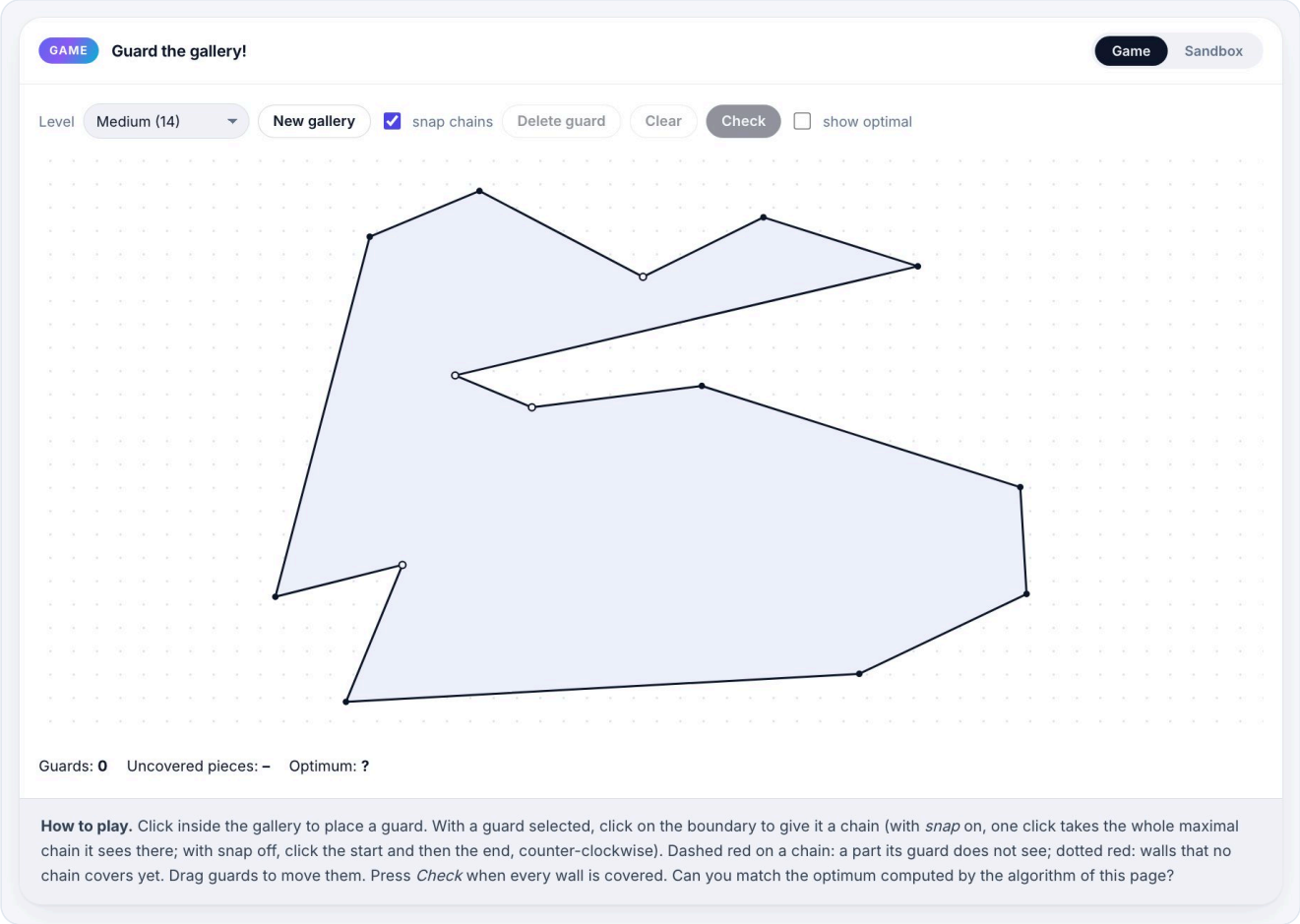
Figure 11 — La fonction next Chapitre 5, §2.10



Ce qu'elle montre. Le graphe de $x \mapsto \mathrm{next}(x)$, ses paliers et ses parties courbes, et la bande des bons départs (vert) ; les marques de U_P tombent dans le vert.

Comment s'en servir. Cliquer dans le graphe pour choisir x_0 : sa révolution est dessinée en escalier et sur le polygone.

Le jeu et le bac à sable Page Play



Ce qu'elle montre. Galerie aléatoire : placer des caméras, choisir leurs murs, vérifier ; le jeu compare avec l'optimum (confettis si c'est optimal). Le bac à sable permet de dessiner ses polygones.

Comment s'en servir. Cliquer dans la galerie pour une caméra, cliquer un mur pour sa portion, puis « Check ».

F

L'implémentation et sa validation

Comment le site et le solveur sont construits, testés et rendus robustes.

1. Lancer, tester, publier

```
cd contiguous-art-gallery
npm install          # une seule fois
npm run dev          # serveur local : http://localhost:5173
npm test             # tests (solveur contre force brute + lemmes)
npm run build        # site statique dans dist/
npm run build:single # le site en UN seul fichier HTML (dist-single/index.html, s'ouvre par double-clic)
npm run docs         # régénère les PDF de docs/ (nécessite Google Chrome)
npm run zip          # fabrique l'archive à envoyer (sources + aperçu + PDF)
```

GitHub Pages. Pousser le dossier sur un dépôt GitHub (branche `main`), puis dans *Settings → Pages* choisir *Source : GitHub Actions*. Le fichier `.github/workflows/deploy.yml` installe, teste, construit et publie le site à chaque push. Le site utilise des chemins relatifs (`base: './'` dans `vite.config.js`) et fonctionne donc aussi dans un sous-dossier.

Technologies. React 18 pour l'interface, Vite 5 pour le build, KaTeX pour les formules, polices Inter / Instrument Serif / JetBrains Mono embarquées (le site marche hors-ligne). Aucune bibliothèque d'animation : les animations sont écrites à la main (CSS + `requestAnimationFrame`) et respectent le réglage « réduire les animations » du système. Toute la géométrie est en JavaScript pur (dossier `src/geometry/`, sans dépendance), ce qui permet de l'exécuter aussi dans Node pour les tests.

2. Architecture du code

Fichier	Rôle
`geometry/vec.js`	vecteurs, test d'orientation, intersection de droites
`geometry/polygon.js`	classe `Polygon` : paramétrisation du bord, appartenance (fermée), prédicat de visibilité `segInside`, lancer de rayon
`geometry/visibility.js`	chaînes maximales vues par un point, polygone de visibilité (dessin)
`geometry/core.js`	noyaux de visibilité (intersection de demi-plans), kernel
`geometry/shortestPath.js`	plus courts chemins dans P (graphe de visibilité + Floyd–Warshall)
`geometry/arrangement.js`	sommets de l'arrangement des droites support à l'intérieur de P
`geometry/solver.js`	l'algorithme : G_P, U_P, $\mathrm{next}(u)$ (lemme 6.8), glouton, théorème 7.1 ; classification (déf. 4.1)
`geometry/proofs.js`	constructions des preuves : prop. 4.3, lemmes 4.2–4.6, 4.9, 6.2, théorème 5.4 (utilisées par les figures)
`geometry/generate.js`	polygones simples aléatoires (« 2-opt untangling ») pour le jeu
`geometry/presets.js`	polygones d'exemple (en position générale)
`App.jsx`, `router.js`	coque du site (barre de navigation, pied de page) et routage par `#/…` (fonctionne sur GitHub Pages et en fichier local)
`pages/`	les pages : `Home` (présentation orale), `ProofIndex`, `ChapterPage` (5 chapitres), `Play`, `Report`
`anim/`	animations : `motion.jsx` (apparition au défilement, horloge en boucle, compteurs, confettis) et `scenes.jsx` (caméras qui balaient les murs, arrangement, lemme 6.2, mauvais gardien)
`components/`	briques SVG (scène, poignées déplaçables, chaînes…), UI (formules, cadres de figure, stepper)
`widgets/`	une figure interactive par fichier + le jeu
`sections/`	texte détaillé des chapitres de la preuve (`chapters.jsx`) et blocs de mise en page (`blocks.jsx`)
`styles.css`	tout le design (thème clair/sombre, typographies Inter et Instrument Serif, animations CSS)
`tests/`	`run-tests.mjs` (solveur), `proofs-test.mjs` (lemmes)
`scripts/`	génération des figures et des PDF de `docs/`

3. Conventions géométriques

- **Orientation.** Les sommets sont stockés dans le sens trigonométrique (le constructeur inverse l'ordre si l'aire signée est négative). Les calculs utilisent des coordonnées mathématiques (y vers le haut) ; le dessin SVG retourne l'axe y, si bien qu'un polygone anti-horaire apparaît aussi anti-horaire à l'écran.
- **Paramétrisation du bord.** Un point du bord est un réel t : $\lfloor t \rfloor$ est l'indice de l'arête et $t - \lfloor t \rfloor$ la fraction parcourue. t et $t + n$ désignent le même point ; une chaîne $[a, b]$ est stockée « déroulée » avec $a \le b \le a + n$. Les sommets ont des paramètres entiers, ce qui rend exacts les tests « la chaîne contient-elle un sommet ».
- **P est fermé.** « g voit q » signifie que le segment gq est dans P *fermé* : il peut longer une arête ou frôler un sommet réflexe. C'est indispensable, car les gardiens candidats sont sur les droites support des arêtes et voient leurs arêtes « par la tranche ».

4. Les briques géométriques

4.1 Le prédicat de visibilité `segInside(p, q)`

On collecte tous les paramètres où le segment pq touche le bord (sommets situés sur pq, intersections, extrémités de chevauchements colinéaires). Une traversée *propre* d'une arête (les deux extrémités de l'arête strictement de part et d'autre de pq et p, q strictement de part et d'autre de l'arête) rend immédiatement « non ». Sinon on teste le milieu de chaque sous-segment entre deux contacts consécutifs : il doit être dans P fermé. Coût $O(n)$ par contact, donc $O(n)$ à $O(n^2)$.

4.2 Chaînes maximales vues par un point `visibleChains(P, g)`

Le long du bord, la visibilité depuis g ne peut changer qu'en un sommet de P ou en un point touché par un rayon issu de g passant par un sommet. On calcule tous ces *points critiques* ($O(n)$ rayons, toutes leurs intersections avec le bord), on teste la visibilité de chaque point critique et du milieu de chaque intervalle élémentaire : entre deux points critiques consécutifs, la visibilité est constante, donc le résultat est exact. On ferme ensuite l'ensemble vu (les extrémités d'un intervalle vu sont vues) et on regroupe les intervalles consécutifs en chaînes maximales. Coût $O(n^2)$ à $O(n^3)$ par point.

4.3 Noyaux de visibilité

Intersection de demi-plans par découpages successifs (Sutherland–Hodgman) d'un très grand carré. Les sommets créés par le carré sont marqués « artificiels » : ils représentent l'infini (noyau non borné) et ne sont jamais utilisés comme candidats.

4.4 Plus courts chemins

Un plus court chemin dans un polygone simple est une ligne brisée dont les sommets intérieurs sont des sommets réflexes. On précalcule quels sommets se voient et on applique Floyd–Warshall (en ne passant que par les sommets réflexes) : $O(n^3)$. Pour une requête entre deux points quelconques, on calcule les sommets vus par chacun et on minimise $|p\,v_i| + D[i][j] + |v_j\,q|$. Le papier utilise Guibas–Hershberger ($O(\log n)$ par requête) ; pour quelques dizaines de sommets la version simple suffit.

4.5 Arrangement

Les $\binom{n}{2}$ intersections de droites support, filtrées pour ne garder que celles dans P, puis fusionnées quand elles coïncident numériquement (en gardant l'union des arêtes qui les définissent).

5. Le solveur (`solver.js`)

1. `isStarShaped()` : noyau de toutes les arêtes ; s'il est non vide, $k = 1$ et le gardien est le centre du noyau.
2. `candidateGuards()` $= G_P$ (déf. 5.1) : pour chaque sommet c de l'arrangement et chaque extrémité x de ses arêtes définissantes vue par c, la chaîne maximale de c contenant x.
3. `startPoints()` $= U_P$ (déf. 5.2) : sommets de P + débuts des chaînes de G_P, triés et sans doublons.
4. `next(u)` (lemme 6.8) : le maximum de
   - `nextGood(u)` : la fin la plus lointaine parmi les chaînes de G_P qui contiennent u ;
   - `nextBad(u)` : pour chaque arête e_j après u (balayage linéaire au lieu de la recherche binaire), plus court chemin de u vers les extrémités de e_j ; s'il tourne à gauche, droite ℓ_u par sa première arête, intersection avec le noyau P_E (maintenu incrémentalement en ajoutant une arête à la fois), et les deux extrémités de $\ell_u \cap P_E$ sont testées comme gardiens (lemme 4.9). On ignore les arêtes qui ne peuvent pas battre la meilleure valeur, et on s'arrête dès que le noyau devient vide (observation 2.1 : plus personne ne peut voir au-delà).

   Les résultats sont mémorisés (`Map`) : deux gloutons qui passent par le même point ne recalculent rien.
5. `greedy(u)` : applique `next` jusqu'à couvrir un tour complet.
6. `solve()` : glouton depuis chaque $u \in U_P$, garde le minimum. Arrêt anticipé : si une solution de taille (taille du premier glouton -1) est trouvée, elle est optimale (borne $k+1$) ; si $k = 2$ est atteint, c'est optimal puisque P n'est pas étoilé. Option `exhaustive` pour la figure 10.

Propriété importante. Toute valeur proposée par `next` vient d'un vrai calcul de visibilité depuis un vrai point : le solveur ne peut donc jamais annoncer une couverture fausse. Le théorème de l'article garantit qu'il ne rate pas l'optimum ; nos tests le vérifient empiriquement.

Performances (navigateur ou Node, polygones aléatoires) : $n = 12$: 9 ms ($k = 2$) ; $n = 20$: 25 ms ($k = 3$) ; $n = 30$: 129 ms ($k = 4$) ; $n = 40$: 324 ms ($k = 8$).

6. Les constructions des preuves (`proofs.js`)

Fonction	Résultat de l'article	Ce qu'elle rend
`replaceGoodGuard`	Prop. 4.3, lemmes 4.2–4.6, cor. 4.7	P_E, P_g, lemme applicable, candidats testés, remplaçant c
`lemma49`	Lemme 4.9	S_v, ℓ_u, ℓ_v, P_v, P', points g^*
`maximalChainLemma62`	Déf. 6.1, lemme 6.2	$E_{g,m}$, s, t, π_s, π_t, rayons, $[u, v]$ et la référence calculée directement
`theorem54`	Lemme 5.3, théorème 5.4	la suite des étapes de la preuve appliquée à une solution
`classifyGuard` (solver.js)	Déf. 4.1	les quatre conditions et l'angle

7. Validation

Les tests (`npm test`) comparent l'implémentation à des calculs indépendants :

- **Visibilité** : pour des points aléatoires, l'appartenance aux chaînes calculées coïncide avec le test direct `segInside`.
- $\mathrm{next}(u)$ **contre force brute** : une grille dense de gardiens ($\approx$ 2000 points + sommets de l'arrangement) ne fait jamais mieux que notre $\mathrm{next}(u)$.
- **Optimum** : le glouton depuis 200 départs régulièrement espacés ne trouve jamais moins de gardiens que l'optimum calculé via U_P ; aucun glouton n'utilise plus de $k + 1$ gardiens ; chaque solution rendue couvre le bord et chaque gardien voit sa chaîne.
- **Lemmes** : bons gardiens : 8647/8647 remplacements trouvés (prop. 4.3, cor. 4.7) ; mauvais gardiens : 1381/1381 (lemme 4.9) ; lemme 6.2 : 5014/5014 égal au calcul direct ; théorème 5.4 : 61/61 déroulés terminés dans U_P.

Dernière exécution : **1541/1541 checks passed**.

8. Robustesse numérique : les pièges rencontrés

1. **Visibilité « par la tranche ».** Un sommet de l'arrangement est sur la droite support de ses arêtes : sans P fermé et sans gestion des contacts, il ne verrait pas ses propres arêtes.
2. **Lemme 6.2, double intersection.** Quand la chaîne fait plus d'un tour autour de g, un rayon peut toucher $[s, t]$ deux fois ; il faut prendre le point le plus proche de m de chaque côté (définition de u^* dans la preuve), pas le plus proche de g. Notre première version se trompait dans $\approx$ 19 % des cas.
3. **Bouclage des paramètres.** Un paramètre $a = 8{,}999999999999998$ comparé à 9 « modulo n » peut donner presque n. Toutes les comparaisons de chaînes passent par `relFrom` qui ramène l'écart dans $(-\varepsilon, n - \varepsilon]$.
4. **Angle d'un gardien.** Si u, g et v sont alignés (la chaîne fait exactement un tour), l'angle entre gu et gv vaut numériquement 0 alors que l'angle balayé vaut 2π : on utilise alors l'angle balayé le long de la chaîne.
5. **Position générale.** L'article suppose qu'aucune paire de droites n'est parallèle et qu'aucun triplet n'est concourant ; les exemples sont légèrement perturbés. Le solveur fonctionne aussi sur des polygones orthogonaux dessinés à la main (droites parallèles ignorées, points concourants fusionnés).

9. Les widgets

Figure	Fichier	Résultat illustré
1	Playground.jsx (accueil)	définition, chaînes maximales d'un point, solution optimale
2	CoreWidget.jsx	noyau P_E, obs. 2.1, 2.2
3	GreedyWidget.jsx	glouton pas à pas, bande « le glouton reste en avance », k vs $k+1$
4	GoodBadWidget.jsx	déf. 4.1 (quatre conditions en direct), obs. 4.8
5	ReplaceWidget.jsx	prop. 4.3, lemmes 4.2–4.6, cor. 4.7 ; lemme 4.9 pour les mauvais gardiens
6	ArrangementWidget.jsx	C_P, G_P, U_P (cliquer un sommet)
7	Theorem54Widget.jsx	théorème 5.4 pas à pas (cas « tous mauvais » sur « Two bad guards »)
8	Lemma62Widget.jsx	lemme 6.2 pas à pas, comparaison avec la visibilité directe
9	NextStepWidget.jsx	lemme 6.8 : partie « bons gardiens » et table des arêtes pour les mauvais
10	FullAlgorithmWidget.jsx	théorème 7.1 : tous les départs de U_P colorés k / $k+1$, polygones aléatoires
11	NextFunctionWidget.jsx	graphe de next, révolutions, zones de bons départs ([6])
Jeu	GameWidget.jsx (page Play)	jeu et bac à sable (dessin, import/export JSON), confettis en cas de victoire

Interaction. Les figures sont en SVG ; les poignées utilisent les *pointer events* (souris et tactile). Les calculs lourds sont mémorisés par polygone (`solverFor`, `solutionFor`) et partagés entre les figures. Chaque sous-section est protégée par une *error boundary* : une erreur dans une figure n'efface pas la page.

10. Modifier le projet

- **Auteurs des sections** : `AUTHORS` dans `src/sections/blocks.jsx` et l'attribut `author` de chaque `<Sub>`.
- **Ajouter un polygone d'exemple** : une entrée dans `src/geometry/presets.js` (coordonnées y vers le haut, environ 0–1000), puis l'ajouter aux listes `ids` des figures voulues.
- **Texte** : accueil `src/pages/Home.jsx`, chapitres `src/sections/chapters.jsx` (métadonnées dans `src/pages/ChapterPage.jsx`), rapport `src/pages/Report.jsx`. Formules : `<M>` (en ligne) ; avec des accolades, écrire `<M>{'\\frac{a}{b}'}</M>`.
- **Régénérer les PDF** : `npm run docs` (nécessite Google Chrome). Les captures d'écran viennent de `scripts/screenshots.mjs` (serveur lancé + Chrome) puis `scripts/trim-shots.py` (Pillow).

G

Continuer le projet

Pour reprendre le travail à partir de l'archive.

G.1 Voir le site tout de suite

Dans l'archive, ouvrir `VOIR_LE_SITE.html` (double-clic) : c'est tout le site dans un seul fichier, sans installation ni connexion.

G.2 Travailler sur le code

1. Installer Node.js 18 ou plus récent.
2. Dans le dossier `projet/` : `npm install` (une seule fois), puis `npm run dev` et ouvrir l'adresse affichée (`http://localhost:5173`). Chaque modification de fichier se voit immédiatement.
3. Avant de partager : `npm test` (tous les tests doivent passer) et `npm run build`.

Je veux…	Fichier
changer le texte de l'accueil (l'oral)	`src/pages/Home.jsx`
changer le texte d'un chapitre	`src/sections/chapters.jsx` ; titres et résumés : `src/pages/ChapterPage.jsx`
changer le rapport	`src/pages/Report.jsx`
changer qui explique quoi	`src/sections/blocks.jsx` et les attributs `author`
changer une figure	`src/widgets/…Widget.jsx`
changer une animation	`src/anim/scenes.jsx`, `src/anim/motion.jsx`
changer les couleurs / le design	`src/styles.css` (variables en haut du fichier)
ajouter un polygone d'exemple	`src/geometry/presets.js`
toucher à l'algorithme	`src/geometry/solver.js` (puis `npm test`)

G.3 Publier sur GitHub Pages

1. Créer un dépôt GitHub et y pousser le contenu de `projet/` (branche `main`).
2. Dans *Settings* → *Pages*, choisir *Source : GitHub Actions*.
3. Le fichier `.github/workflows/deploy.yml` teste, construit et publie le site à chaque push ; l'adresse apparaît dans l'onglet *Actions*.

G.4 Régénérer les documents et l'archive

- `npm run docs` : régénère ce PDF et les brouillons (nécessite Google Chrome).
- Captures d'écran : lancer `npm run dev`, puis `node scripts/screenshots.mjs http://localhost:5173` et `python3 scripts/trim-shots.py docs/screenshots` (Pillow).
- `npm run build:single` : l'aperçu en un seul fichier ; `npm run zip` : l'archive complète.

G.5 Ce qu'il reste à faire

- Relire chaque chapitre face à l'article et reformuler avec nos mots ce que nous dirons à l'oral.
- Valider (ou changer) la répartition des explications, et compléter le document « Work split » (docs/).
- Compléter le journal IA (docs/) avec notre propre évaluation : il doit rester exact.
- Publier le site sur GitHub Pages et mettre le lien dans le rapport.
- Répéter l'oral avec le déroulé C.1 (chronométrer).